

**BỘ KHOA HỌC VÀ CÔNG NGHỆ**  
**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG**



**BÁO CÁO BÀI TẬP**

**ASSIGNMENT 4.2**

**GVHD: TRẦN ĐÌNH QUẾ**

**LỚP CQ 03 – NHÓM 01**

**Họ và tên:** Đỗ Ngọc Lâm

**Mã sinh viên:** B22DCCN476

**Lớp:** D22CNPM04

Hà Nội, 09/2025

## MỤC LỤC

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>DANH MỤC HÌNH ẢNH .....</b>                  | <b>3</b>  |
| <b>INSTRUCTIONS.....</b>                        | <b>4</b>  |
| ⇒ <b>Convert dataset from .txt to .csv.....</b> | <b>5</b>  |
| ⇒ <b>Câu b → g .....</b>                        | <b>9</b>  |
| ⇒ <b>Kết quả.....</b>                           | <b>16</b> |

## DANH MỤC HÌNH ẢNH

|               |    |
|---------------|----|
| Hình 1 .....  | 5  |
| Hình 2 .....  | 5  |
| Hình 3 .....  | 6  |
| Hình 4 .....  | 6  |
| Hình 5 .....  | 7  |
| Hình 6 .....  | 7  |
| Hình 7 .....  | 8  |
| Hình 8 .....  | 8  |
| Hình 9 .....  | 9  |
| Hình 10 ..... | 9  |
| Hình 11 ..... | 10 |
| Hình 12 ..... | 10 |
| Hình 13 ..... | 11 |
| Hình 14 ..... | 11 |
| Hình 15 ..... | 12 |
| Hình 16 ..... | 12 |
| Hình 17 ..... | 13 |
| Hình 18 ..... | 13 |
| Hình 19 ..... | 14 |
| Hình 20 ..... | 14 |
| Hình 21 ..... | 15 |
| Hình 22 ..... | 16 |
| Hình 23 ..... | 16 |
| Hình 24 ..... | 17 |
| Hình 25 ..... | 17 |
| Hình 26 ..... | 18 |
| Hình 27 ..... | 18 |
| Hình 28 ..... | 19 |
| Hình 29 ..... | 19 |
| Hình 30 ..... | 20 |
| Hình 31 ..... | 20 |
| Hình 32 ..... | 21 |
| Hình 33 ..... | 21 |
| Hình 34 ..... | 22 |
| Hình 35 ..... | 22 |
| Hình 36 ..... | 23 |
| Hình 37 ..... | 23 |
| Hình 38 ..... | 24 |

## INSTRUCTIONS

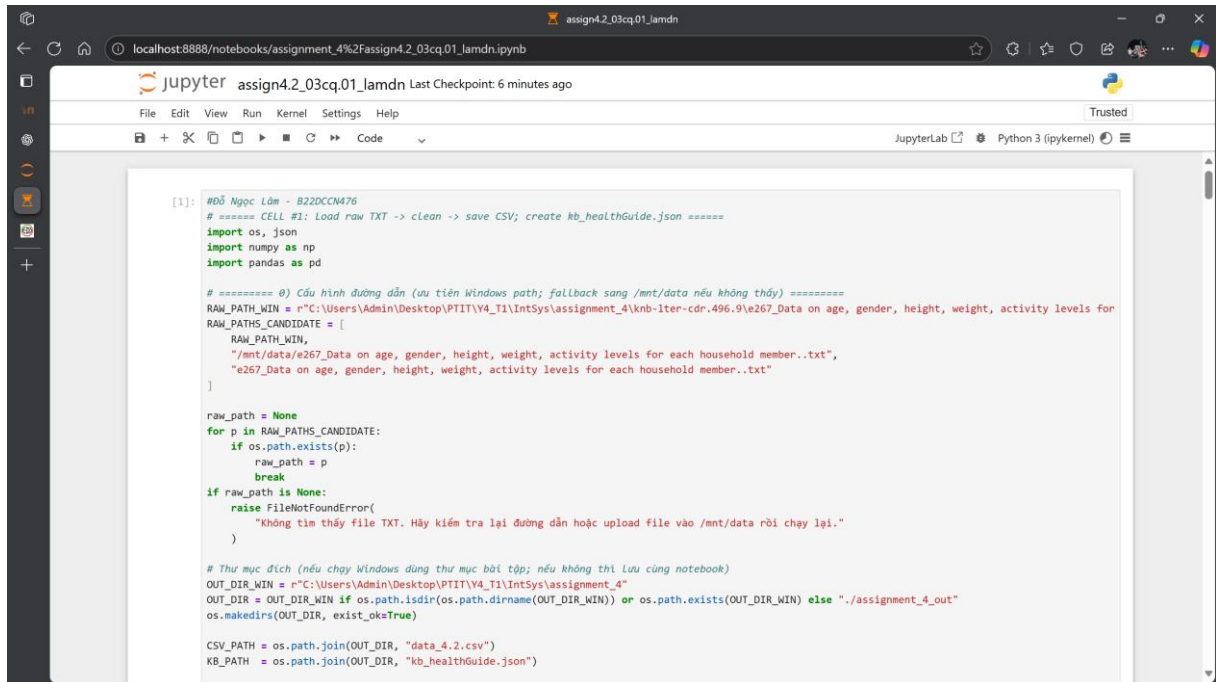
### 4.2. The same as ex 4.1 but with provided datasets.

- a. ~~Build a dataset with 1000 persons and 4 features (jobs, age, height, weight) and store in C:\DATA\data\_4.1.csv~~
- b. Show the distribution of the dataset
- c. Using 5 basic ML models and using BMI to classifying and predicting overweight, underweight, normal with respect to age and job
- d. Build models CNN, RNN, LSTM with 5 layers for classification and prediction problem in c
- e. Compare and evaluate models given in c. and d. with metrics accuracy, MAE, MSE, RMSE
- f. Visualize with  $\geq 5$  various types
- g. Deploy the best model so that user enters: job, age, height, weight and output is underweight, overweight or normal and give a guide for health.

#### Note:

- Code b,c,d,e,f,g must be put and run in ONE CELL
- In order to make a health-guide in Question d, student must build a knowledge base from web (for simple using JSON) and store in C:\DATA\kb\_healthGuide.json

## ⇒ Convert dataset from .txt to .csv



```
[1]: #Đỗ Ngọc Lâm - B22DCCN476
# ===== CELL #1: Load raw TXT -> clean -> save CSV; create kb_healthGuide.json =====
import os, json
import numpy as np
import pandas as pd

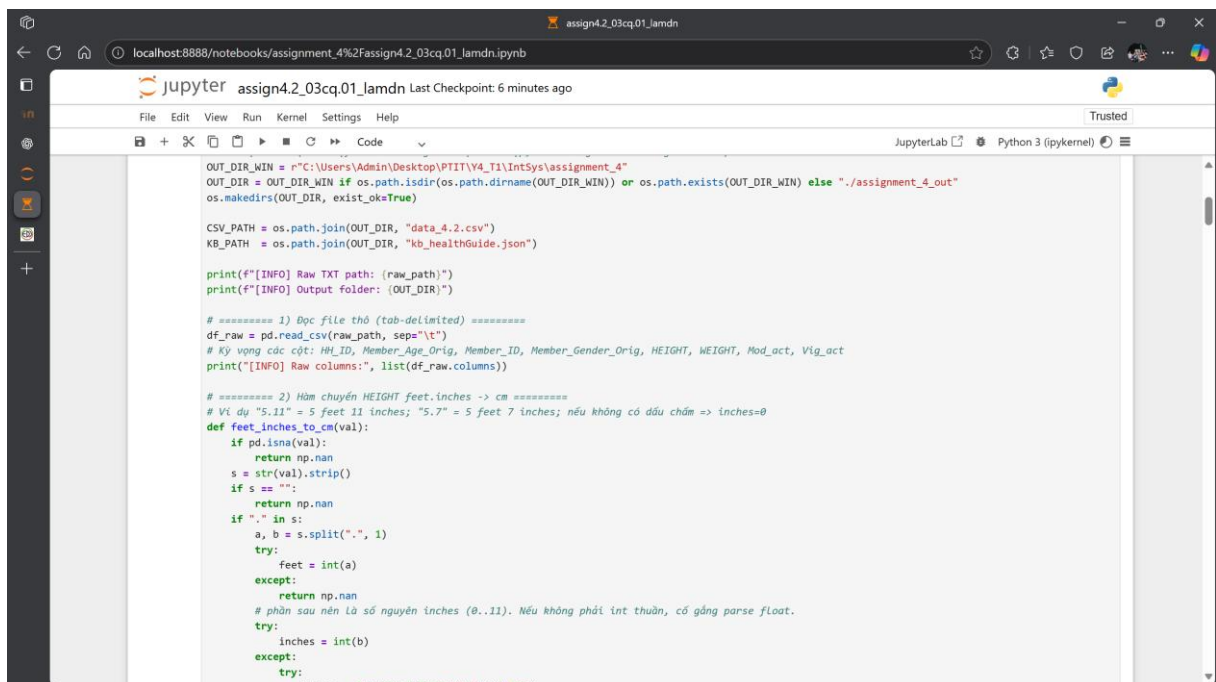
# ===== 0) Cấu hình đường dẫn (ưu tiên Windows path; fallback sang /mnt/data nếu không thấy) =====
RAW_PATH_WIN = r"C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4\knb-iter-cdr.496.9\267_Data on age, gender, height, weight, activity levels for
RAW_PATHS_CANDIDATE = [
    RAW_PATH_WIN,
    "/mnt/data/e267_Data on age, gender, height, weight, activity levels for each household member.txt",
    "e267_Data on age, gender, height, weight, activity levels for each household member.txt"
]

raw_path = None
for p in RAW_PATHS_CANDIDATE:
    if os.path.exists(p):
        raw_path = p
        break
if raw_path is None:
    raise FileNotFoundError(
        "Không tìm thấy file TXT. Hãy kiểm tra lại đường dẫn hoặc upload file vào /mnt/data rồi chạy lại."
    )

# Thư mục đích (nếu chạy Windows dùng thư mục bài tập; nếu không thì lưu cùng notebook)
OUT_DIR_WIN = r"C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4"
OUT_DIR = OUT_DIR_WIN if os.path.isdir(os.path.dirname(OUT_DIR_WIN)) or os.path.exists(OUT_DIR_WIN) else "./assignment_4_out"
os.makedirs(OUT_DIR, exist_ok=True)

CSV_PATH = os.path.join(OUT_DIR, "data_4.2.csv")
KB_PATH = os.path.join(OUT_DIR, "kb_healthGuide.json")
```

Hình 1



```
OUT_DIR_WIN = r"C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4"
OUT_DIR = OUT_DIR_WIN if os.path.isdir(os.path.dirname(OUT_DIR_WIN)) or os.path.exists(OUT_DIR_WIN) else "./assignment_4_out"
os.makedirs(OUT_DIR, exist_ok=True)

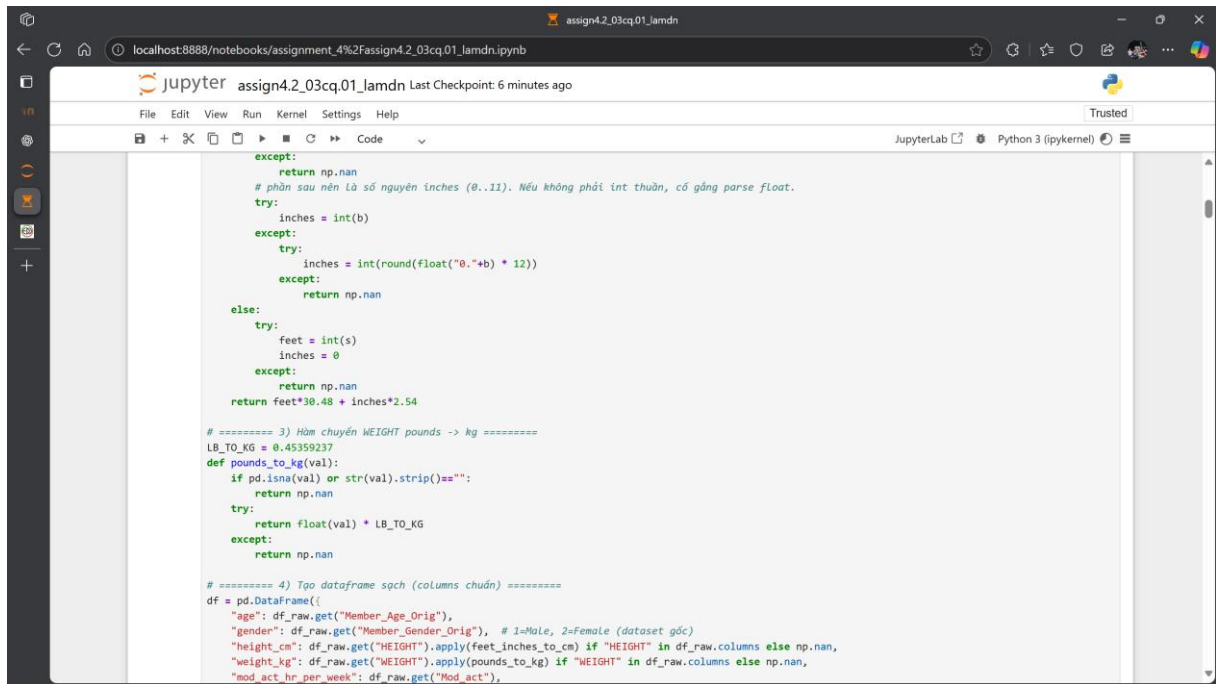
CSV_PATH = os.path.join(OUT_DIR, "data_4.2.csv")
KB_PATH = os.path.join(OUT_DIR, "kb_healthGuide.json")

print(f"[INFO] Raw TXT path: {raw_path}")
print(f"[INFO] Output folder: {OUT_DIR}")

# ===== 1) Đọc file thô (tab-delimited) =====
df_raw = pd.read_csv(raw_path, sep="\t")
# Kỳ vọng các cột: Ht_ID, Member_Age_Orig, Member_ID, Member_Gender_Orig, HEIGHT, WEIGHT, Mod_act, Vig_act
print(f"[INFO] Raw columns:", list(df_raw.columns))

# ===== 2) Hàm chuyển HEIGHT feet.inches -> cm =====
# Ví dụ "5.11" = 5 feet 11 inches; "5.7" = 5 feet 7 inches; nếu không có dấu chấm => inches=0
def feet_inches_to_cm(val):
    if pd.isna(val):
        return np.nan
    s = str(val).strip()
    if s == "":
        return np.nan
    if "." in s:
        a, b = s.split(".", 1)
        try:
            feet = int(a)
        except:
            return np.nan
        # phần sau nên là số nguyên inches (0..11). Nếu không phải int thuần, cố gắng parse float.
        try:
            inches = int(b)
        except:
            try:
                inches = float(b)
```

Hình 2

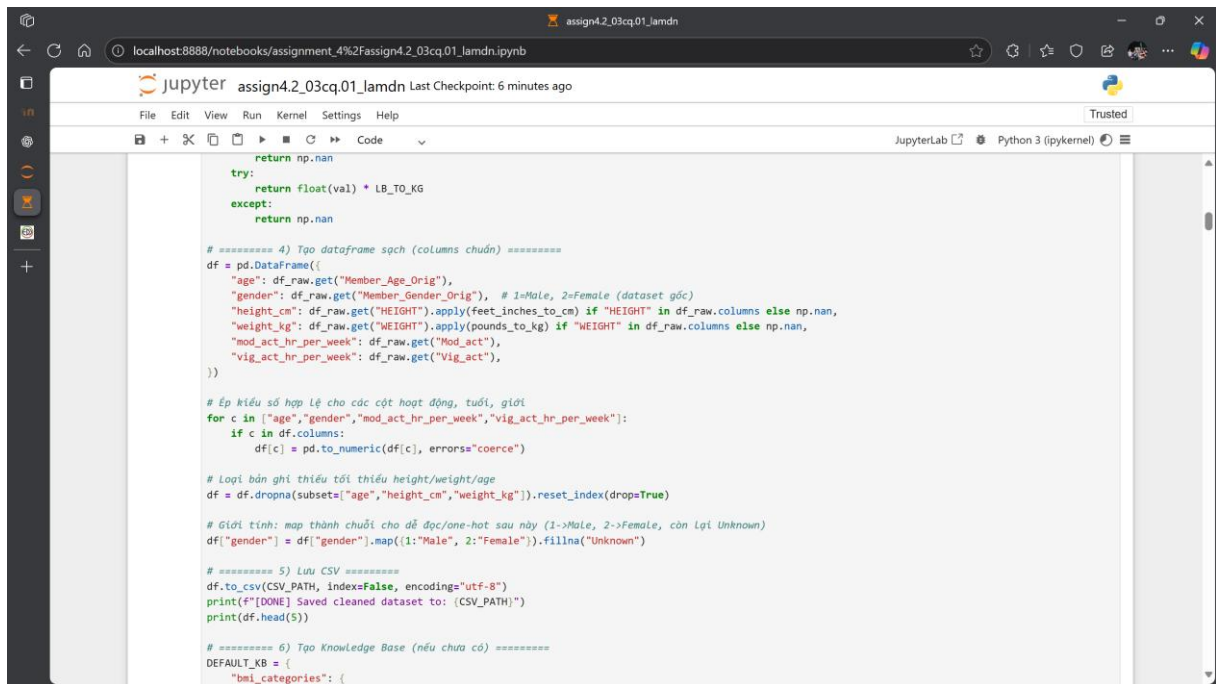


```
except:
    return np.nan
# phần sau nên là số nguyên inches (0..11). Nếu không phải int thuần, cố gắng parse float.
try:
    inches = int(b)
except:
    try:
        inches = int(round(float("0."+b) * 12))
    except:
        return np.nan
else:
    try:
        feet = int(s)
        inches = 0
    except:
        return np.nan
    return feet*30.48 + inches*2.54

# ===== 3) Hàm chuyển WEIGHT pounds -> kg =====
LB_TO_KG = 0.45359237
def pounds_to_kg(val):
    if pd.isna(val) or str(val).strip()=="":
        return np.nan
    try:
        return float(val) * LB_TO_KG
    except:
        return np.nan

# ===== 4) Tạo dataframe sạch (columns chuẩn) =====
df = pd.DataFrame({
    "age": df_raw.get("Member_Age_Orig"),
    "gender": df_raw.get("Member_Gender_Orig"), # 1=Male, 2=Female (dataset gốc)
    "height_cm": df_raw.get("HEIGHT").apply(feet_inches_to_cm) if "HEIGHT" in df_raw.columns else np.nan,
    "weight_kg": df_raw.get("WEIGHT").apply(pounds_to_kg) if "WEIGHT" in df_raw.columns else np.nan,
    "mod_act_hr_per_week": df_raw.get("Mod_act"),
})
```

Hình 3



```
try:
    return float(val) * LB_TO_KG
except:
    return np.nan

# ===== 4) Tạo dataframe sạch (columns chuẩn) =====
df = pd.DataFrame({
    "age": df_raw.get("Member_Age_Orig"),
    "gender": df_raw.get("Member_Gender_Orig"), # 1=Male, 2=Female (dataset gốc)
    "height_cm": df_raw.get("HEIGHT").apply(feet_inches_to_cm) if "HEIGHT" in df_raw.columns else np.nan,
    "weight_kg": df_raw.get("WEIGHT").apply(pounds_to_kg) if "WEIGHT" in df_raw.columns else np.nan,
    "mod_act_hr_per_week": df_raw.get("Mod_act"),
    "vig_act_hr_per_week": df_raw.get("Vig_act"),
})

# Ép kiểu số hợp lệ cho các cột hoạt động, tuổi, giới
for c in ["age", "gender", "mod_act_hr_per_week", "vig_act_hr_per_week"]:
    if c in df.columns:
        df[c] = pd.to_numeric(df[c], errors="coerce")

# Loại bỏ ghi thiếu tối thiểu height/weight/age
df = df.dropna(subset=["age", "height_cm", "weight_kg"]).reset_index(drop=True)

# Giới tính: map thành chuỗi cho dễ đọc/one-hot sau này (1->Male, 2->Female, còn lại Unknown)
df["gender"] = df["gender"].map({1:"Male", 2:"Female"}).fillna("Unknown")

# ===== 5) Lưu CSV =====
df.to_csv(CSV_PATH, index=False, encoding="utf-8")
print(f"[DONE] Saved cleaned dataset to: {CSV_PATH}")
print(df.head(5))

# ===== 6) Tạo Knowledge Base (nếu chưa có) =====
DEFAULT_KB = {
    "bmi_categories": {
```

Hình 4

```
df.to_csv(CSV_PATH, index=False, encoding="utf-8")
print(f"[DONE] Saved cleaned dataset to: {CSV_PATH}")
print(df.head(5))

# ===== 6) Tạo Knowledge Base (nếu chưa có) =====
DEFAULT_KB = {
    "bmi_categories": {
        "underweight": "BMI < 18.5",
        "normal": "18.5 ≤ BMI < 25",
        "overweight": "BMI ≥ 25"
    },
    "advice": {
        "underweight": [
            "Tăng năng lượng lành mạnh: thêm ngũ cốc nguyên hạt, các loại hạt, dầu tốt.",
            "Ăn đủ đạm (thịt nạc, trứng, sữa chua, đậu).",
            "Chia nhỏ bữa, thêm snack giàu dinh dưỡng.",
            "Khám chuyên khoa nếu sụt cân nhanh/khó tăng cân."
        ],
        "normal": [
            "Duy trì khẩu phần cân bằng, nhiều rau quả (≥400 g/ngày).",
            "Hoạt động thể lực ≥150 phút/tuần mức vừa + 2 ngày/tuần tăng cơ.",
            "Hạn chế nước ngọt có đường, đồ siêu chế biến.",
            "Ngủ đủ giấc, quản lý stress."
        ],
        "overweight": [
            "Đặt mục tiêu giảm cân thực tế; theo dõi khẩu phần/calori.",
            "Tăng rau quả; giảm đường, muối, chất béo bão hòa.",
            "Vận động ≥150 phút/tuần + 2 ngày/tuần tăng cơ.",
            "Tư vấn dinh dưỡng y khoa nếu cần."
        ]
    },
    "sources": [
        "WHO/CDC BMI cutoffs; WHO healthy diet; CDC/WHO physical activity"
    ]
}
```

Hình 5

```
        "Vận động ≥150 phút/tuần + 2 ngày/tuần tăng cơ.",
        "Tư vấn dinh dưỡng y khoa nếu cần."
    ],
    "sources": [
        "WHO/CDC BMI cutoffs; WHO healthy diet; CDC/WHO physical activity"
    ]
}

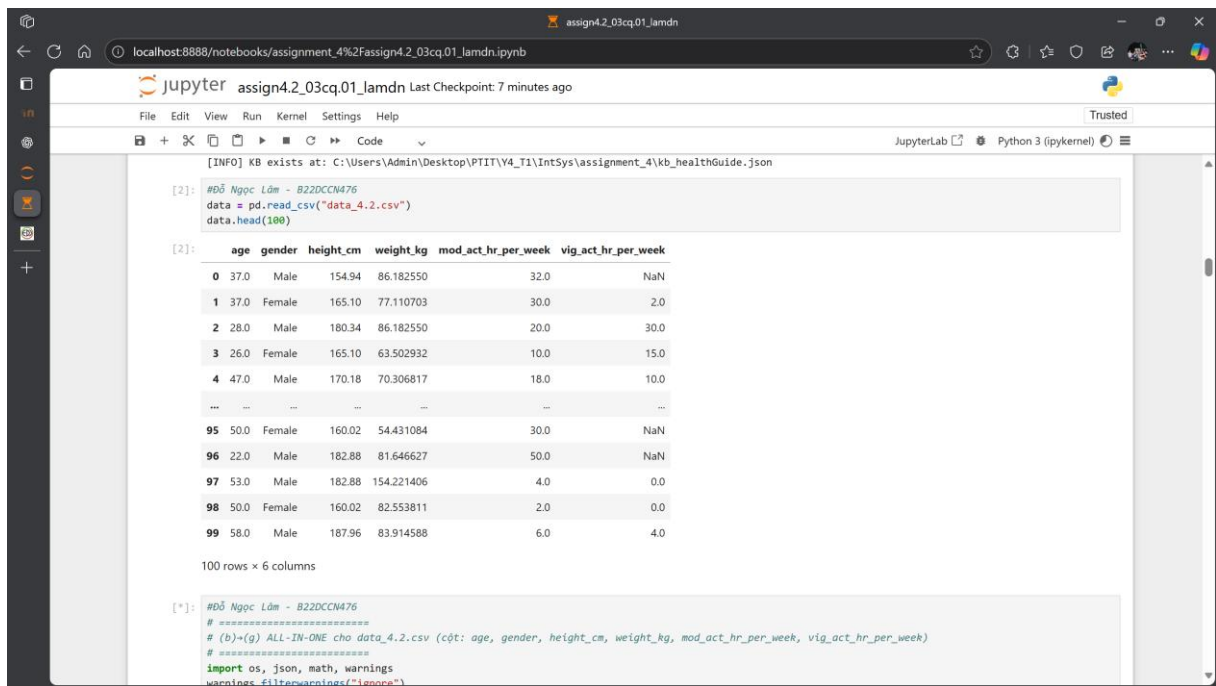
if not os.path.exists(KB_PATH):
    with open(KB_PATH, "w", encoding="utf-8") as f:
        json.dump(DEFAULT_KB, f, ensure_ascii=False, indent=2)
    print(f"[DONE] Created KB at: {KB_PATH}")
else:
    print(f"[INFO] KB exists at: {KB_PATH}")

[INFO] Raw TXT path: C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4\knb-ltr-cdr.496.9\ae267_Data on age, gender, height, weight, activity levels for each household member..txt
[INFO] Output folder: C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4
[INFO] Raw columns: ['HH_ID', 'Member_Age_Orig', 'Member_ID', 'Member_Gender_Orig', 'HEIGHT', 'WEIGHT', 'Mod_act', 'Vig_act']
[INFO] Saved cleaned dataset to: C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4\data_4.2.csv

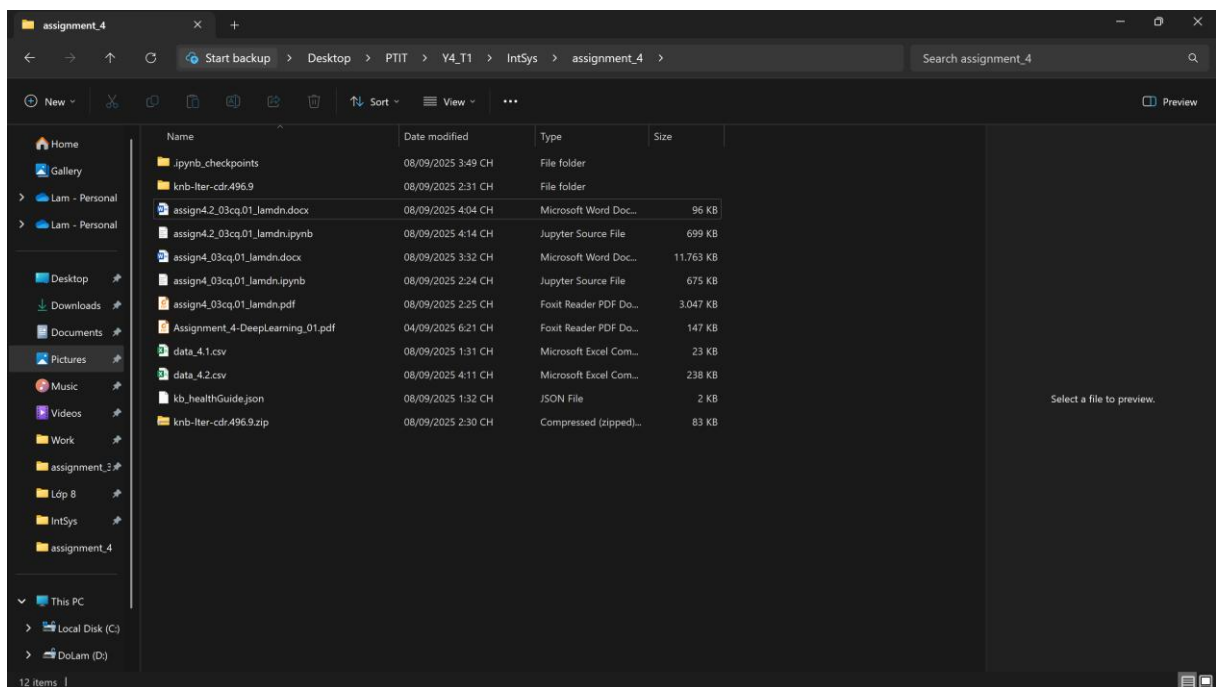
age  gender  height_cm  weight_kg  mod_act_hr_per_week \
0  37.0   Male    154.04    86.182550      32.0
1  37.0   Female   165.10    77.110703      30.0
2  28.0   Male    180.34    86.182550      20.0
3  26.0   Female   165.10    63.502932      10.0
4  47.0   Male    170.18    70.306817      18.0

vig_act_hr_per_week
0      NaN
1       2.0
2      30.0
3      15.0
4      10.0
[INFO] KB exists at: C:\Users\Admin\Desktop\PTIT\V4_T1\IntSys\assignment_4\kb_healthGuide.json
```

Hình 6



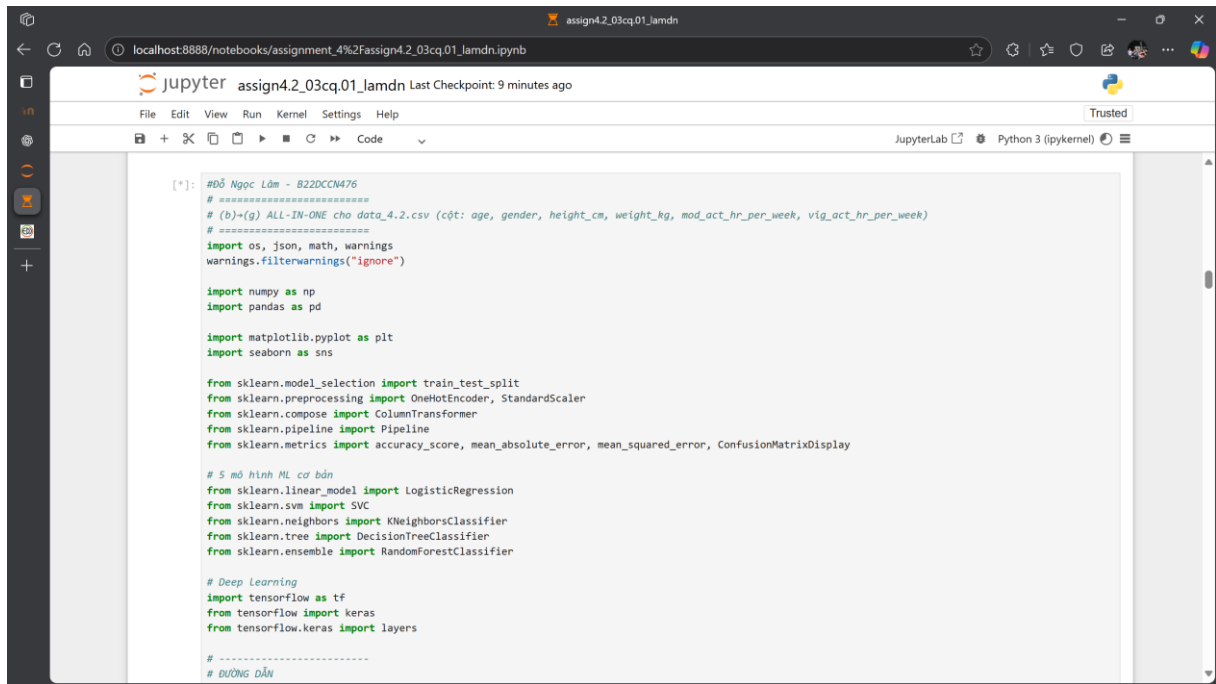
Hình 7



Hình 8



⇒ Câu b → g



```
[*]: #Đỗ Ngọc Lâm - B22DCCN476
# =====
# (b)→(g) ALL-IN-ONE cho data_4.2.csv (cột: age, gender, height_cm, weight_kg, mod_act_hr_per_week, vig_act_hr_per_week)
# =====
import os, json, math, warnings
warnings.filterwarnings("ignore")

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

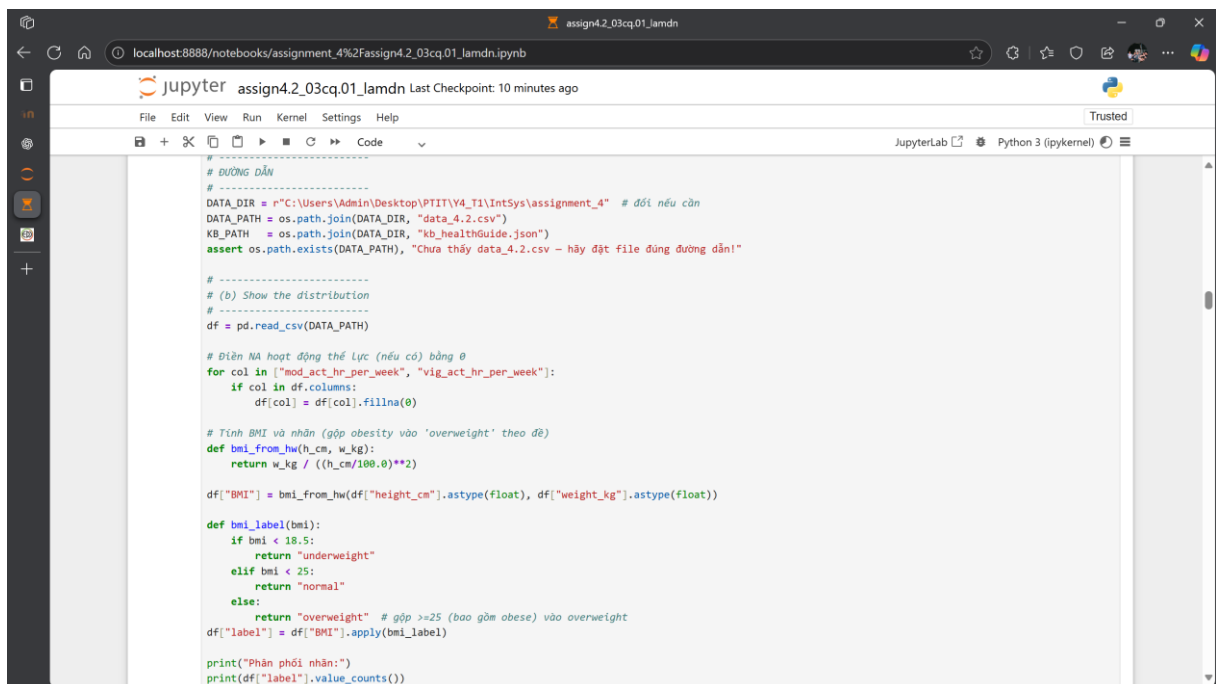
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score, mean_absolute_error, mean_squared_error, ConfusionMatrixDisplay

# 5 mô hình ML cơ bản
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

# Deep Learning
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers

# -----
# ĐƯỜNG DẪN
```

Hình 9



```
# -----
# ĐƯỜNG DẪN
DATA_DIR = r"C:\Users\Admin\Desktop\PTIT\IV4_T1\IntSys\assignment_4" # đổi nếu cần
DATA_PATH = os.path.join(DATA_DIR, "data_4.2.csv")
KB_PATH = os.path.join(DATA_DIR, "kb_healthGuide.json")
assert os.path.exists(DATA_PATH), "Chưa thấy data_4.2.csv - hãy đặt file đúng đường dẫn!"

# -----
# (b) Show the distribution
# -----
df = pd.read_csv(DATA_PATH)

# Điền NA hoạt động thể lực (nếu có) bằng 0
for col in ["mod_act_hr_per_week", "vig_act_hr_per_week"]:
    if col in df.columns:
        df[col] = df[col].fillna(0)

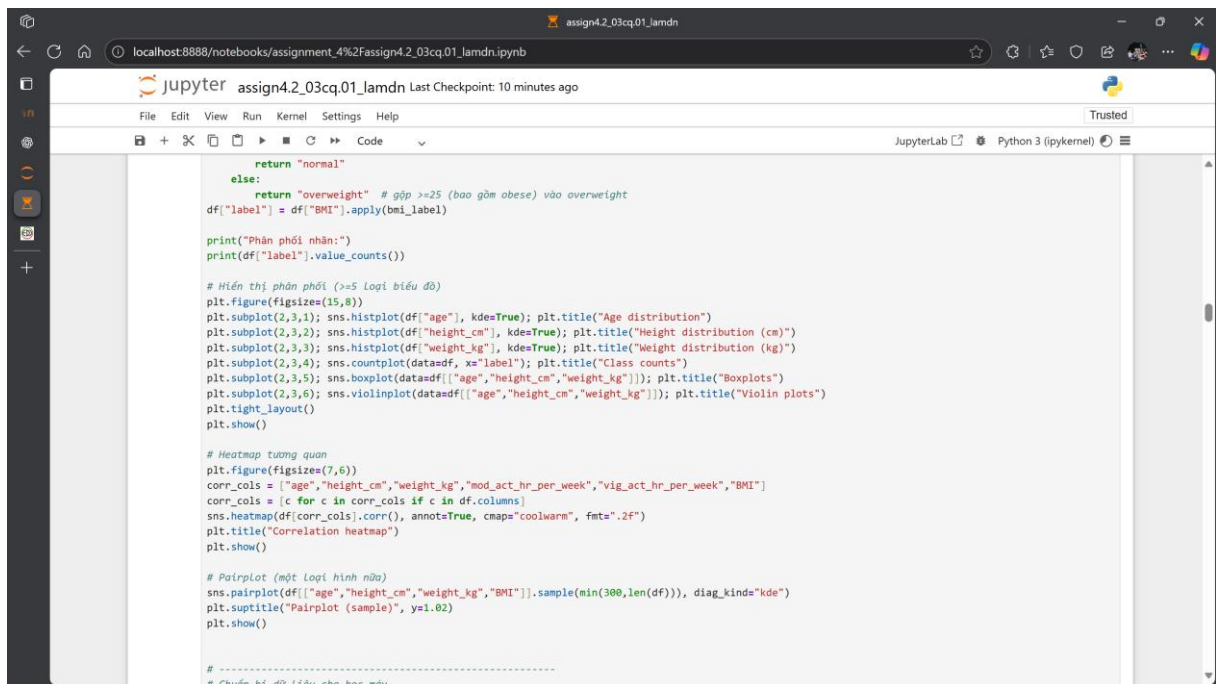
# Tính BMI và nhân (gộp obesity vào 'overweight' theo đề)
def bmi_from_hw(h_cm, w_kg):
    return w_kg / ((h_cm/100.0)**2)

df["BMI"] = bmi_from_hw(df["height_cm"].astype(float), df["weight_kg"].astype(float))

def bmi_label(bmi):
    if bmi < 18.5:
        return "underweight"
    elif bmi < 25:
        return "normal"
    else:
        return "overweight" # gộp >=25 (bao gồm obese) vào overweight
df["label"] = df["BMI"].apply(bmi_label)

print("Phân phối nhân:")
print(df["label"].value_counts())
```

Hình 10



```
        return "normal"
    else:
        return "overweight" # gộp >=25 (bao gồm obese) vào overweight
df["label"] = df["BMI"].apply(bmi_label)

print("Phân phối nhân:")
print(df["label"].value_counts())

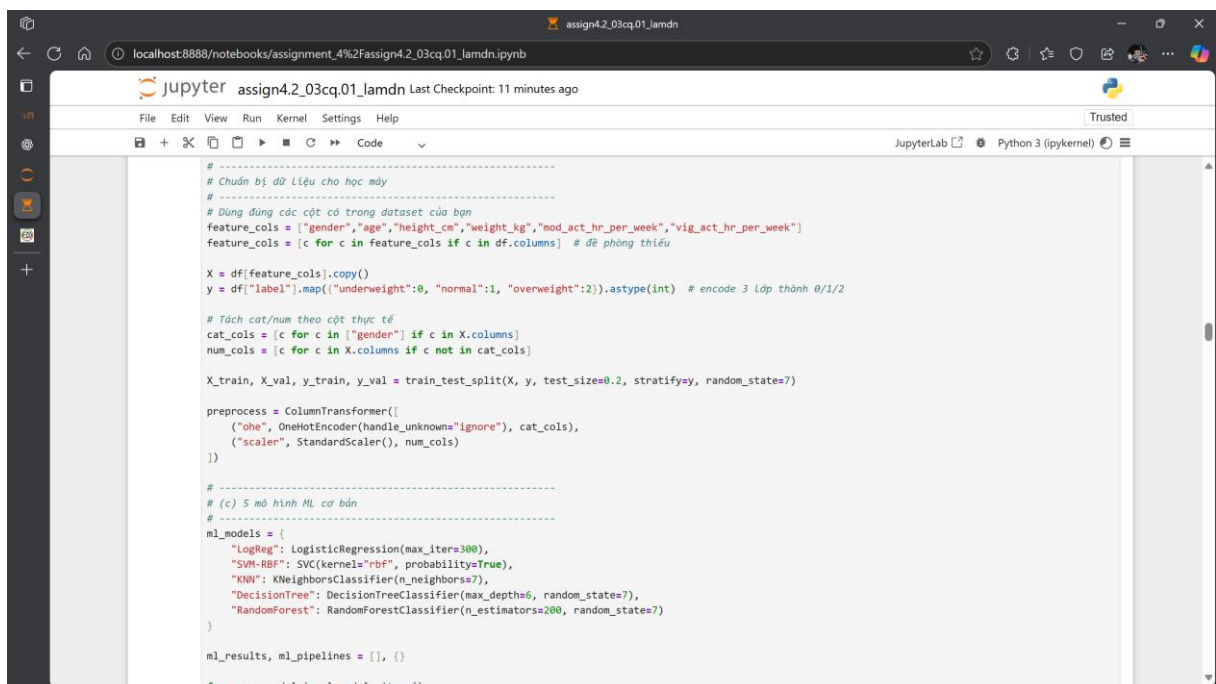
# Hiển thị phân phối (>=5 Loại biểu đồ)
plt.figure(figsize=(15,8))
plt.subplot(2,3,1); sns.histplot(df["age"], kde=True); plt.title("Age distribution")
plt.subplot(2,3,2); sns.histplot(df["height_cm"], kde=True); plt.title("Height distribution (cm)")
plt.subplot(2,3,3); sns.histplot(df["weight_kg"], kde=True); plt.title("Weight distribution (kg)")
plt.subplot(2,3,4); sns.countplot(data=df, x="label"); plt.title("Class counts")
plt.subplot(2,3,5); sns.boxplot(data=df[["age", "height_cm", "weight_kg"]]); plt.title("Boxplots")
plt.subplot(2,3,6); sns.violinplot(data=df[["age", "height_cm", "weight_kg"]]); plt.title("Violin plots")
plt.tight_layout()
plt.show()

# Heatmap tương quan
plt.figure(figsize=(7,6))
corr_cols = ["age", "height_cm", "weight_kg", "mod_act_hr_per_week", "vig_act_hr_per_week", "BMI"]
corr_cols = [c for c in corr_cols if c in df.columns]
sns.heatmap(df[corr_cols].corr(), annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation heatmap")
plt.show()

# Pairplot (một loại hình nãu)
sns.pairplot(df[["age", "height_cm", "weight_kg", "BMI"]].sample(min(300, len(df)), diag_kind="kde"))
plt.suptitle("Pairplot (sample)", y=1.02)
plt.show()

# Chuẩn bị dữ liệu cho học máy
```

Hình 11



```
# -----
# Chuẩn bị dữ liệu cho học máy
# -----
# Dùng đúng các cột có trong dataset của bạn
feature_cols = ["gender", "age", "height_cm", "weight_kg", "mod_act_hr_per_week", "vig_act_hr_per_week"]
feature_cols = [c for c in feature_cols if c in df.columns] # để phòng thiếu

X = df[feature_cols].copy()
y = df["label"].map({"underweight":0, "normal":1, "overweight":2}).astype(int) # encode 3 lớp thành 0/1/2

# Tách cat/num theo cột thực tế
cat_cols = [c for c in ["gender"] if c in X.columns]
num_cols = [c for c in X.columns if c not in cat_cols]

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, stratify=y, random_state=7)

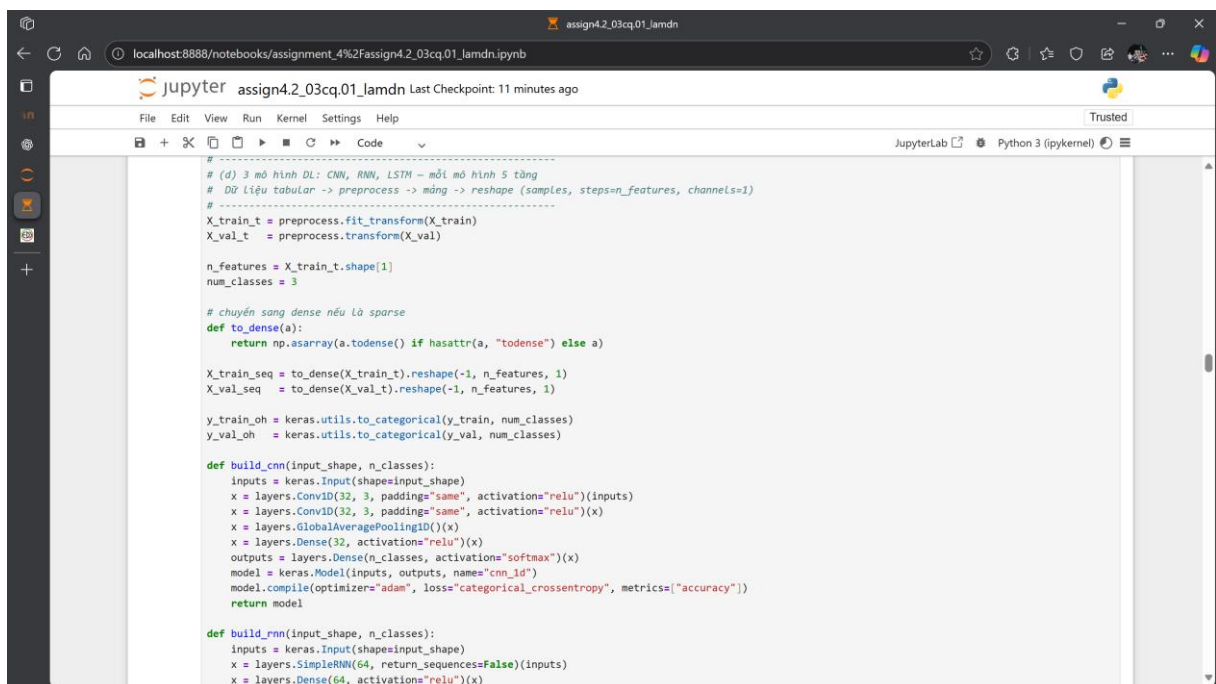
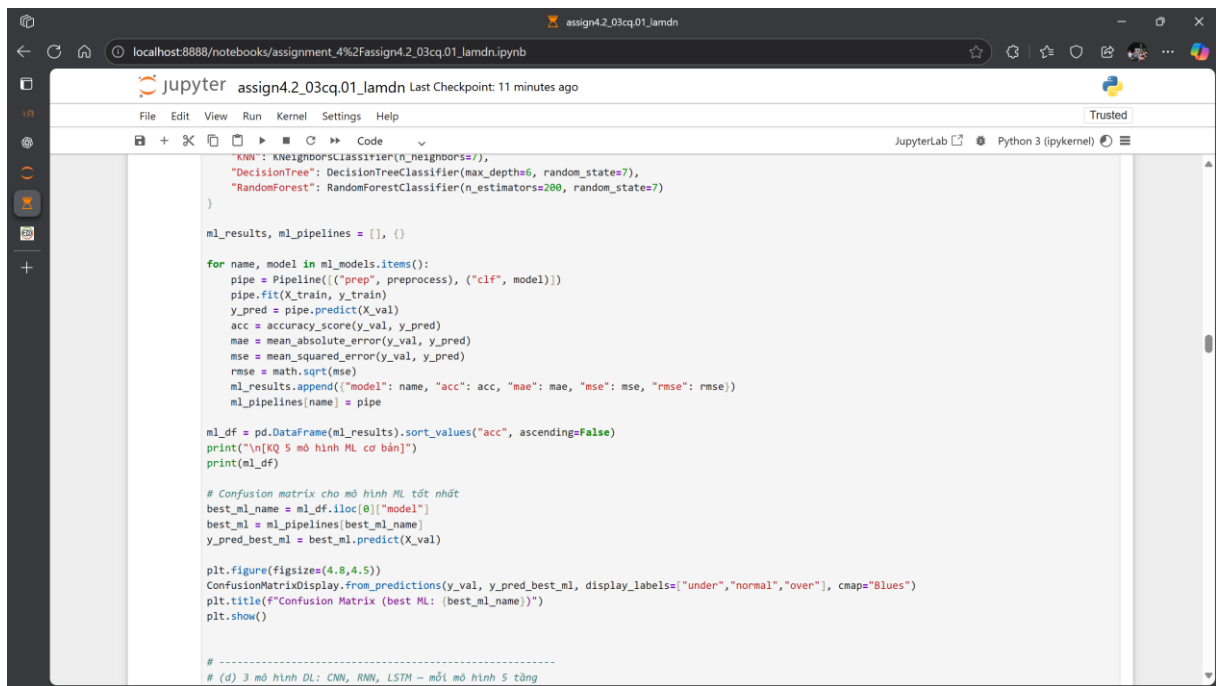
preprocess = ColumnTransformer([
    ("ohe", OneHotEncoder(handle_unknown="ignore"), cat_cols),
    ("scaler", StandardScaler(), num_cols)
])

# -----
# (c) 5 mô hình ML cơ bản
# -----
ml_models = {
    "LogReg": LogisticRegression(max_iter=300),
    "SVH-RBF": SVC(kernel="rbf", probability=True),
    "KNN": KNeighborsClassifier(n_neighbors=7),
    "DecisionTree": DecisionTreeClassifier(max_depth=6, random_state=7),
    "RandomForest": RandomForestClassifier(n_estimators=200, random_state=7)
}

ml_results, ml_pipelines = [], {}

for name, model in ml_models.items():
```

Hình 12



```
model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
return model

def build_rnn(input_shape, n_classes):
    inputs = keras.Input(shape=input_shape)
    x = layers.SimpleRNN(64, return_sequences=False)(inputs)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dense(32, activation="relu")(x)
    outputs = layers.Dense(n_classes, activation="softmax")(x)
    model = keras.Model(inputs, outputs, name="rnn_simple")
    model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
    return model

def build_lstm(input_shape, n_classes):
    inputs = keras.Input(shape=input_shape)
    x = layers.LSTM(64, return_sequences=False)(inputs)
    x = layers.Dense(64, activation="relu")(x)
    x = layers.Dense(32, activation="relu")(x)
    outputs = layers.Dense(n_classes, activation="softmax")(x)
    model = keras.Model(inputs, outputs, name="lstm_1")
    model.compile(optimizer="adam", loss="categorical_crossentropy", metrics=["accuracy"])
    return model

cnn = build_cnn((n_features,1), num_classes)
rnn = build_rnn((n_features,1), num_classes)
lstm = build_lstm((n_features,1), num_classes)

early = keras.callbacks.EarlyStopping(patience=10, restore_best_weights=True, monitor="val_accuracy")

hist_cnn = cnn.fit(X_train_seq, y_train_oh, validation_data=(X_val_seq, y_val_oh),
                  epochs=60, batch_size=64, verbose=0, callbacks=[early])
hist_rnn = rnn.fit(X_train_seq, y_train_oh, validation_data=(X_val_seq, y_val_oh),
                  epochs=60, batch_size=64, verbose=0, callbacks=[early])
hist_lstm = lstm.fit(X_train_seq, y_train_oh, validation_data=(X_val_seq, y_val_oh),
                    epochs=60, batch_size=64, verbose=0, callbacks=[early])
```

Hình 15

```
hist_lstm = lstm.fit(X_train_seq, y_train_oh, validation_data=(X_val_seq, y_val_oh),
                    epochs=60, batch_size=64, verbose=0, callbacks=[early])

def eval_dl(model, name):
    y_pred_prob = model.predict(X_val_seq, verbose=0)
    y_pred = y_pred_prob.argmax(axis=1)
    acc = accuracy_score(y_val, y_pred)
    mae = mean_absolute_error(y_val, y_pred)
    mse = mean_squared_error(y_val, y_pred)
    rmse = math.sqrt(mse)
    return {"model": name, "acc": acc, "mae": mae, "mse": mse, "rmse": rmse}

dl_results = [eval_dl(m, nm) for m, nm in [(cnn, "CNN"), (rnn, "RNN"), (lstm, "LSTM")]]
dl_df = pd.DataFrame(dl_results).sort_values("acc", ascending=False)
print("\n[KQ mô hình DL (CNN/RNN/LSTM)]")
print(dl_df)

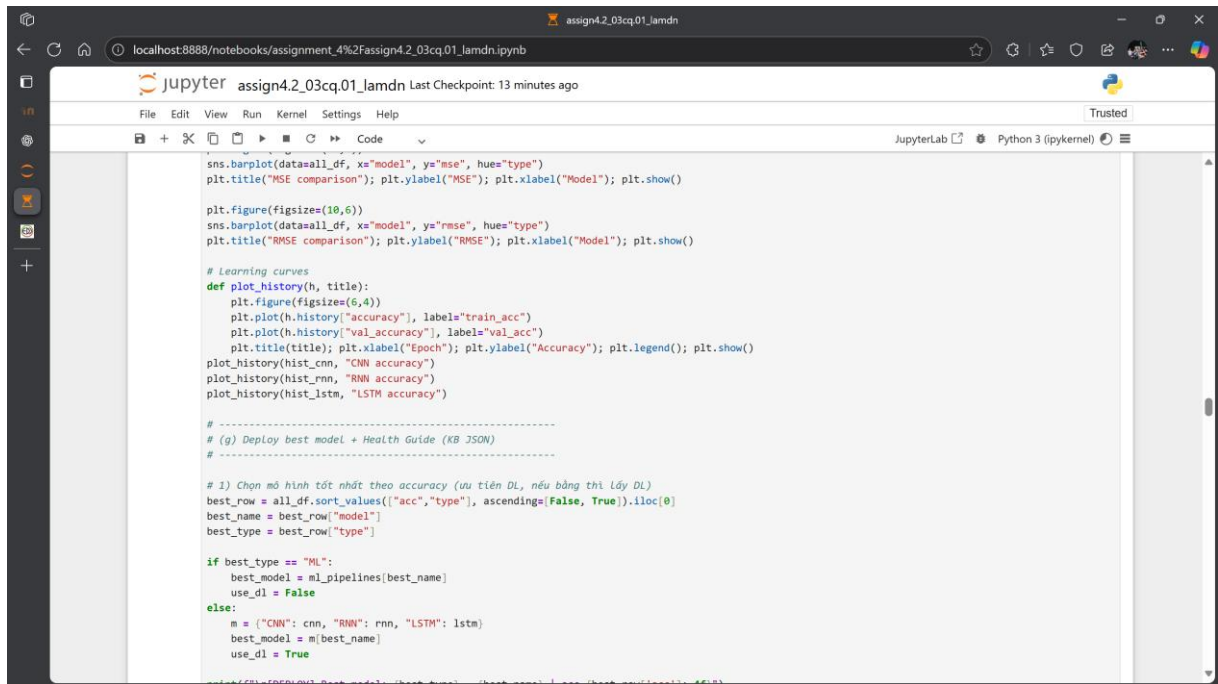
# So sánh tất cả mô hình (c & d)
all_df = pd.concat([ml_df.assign(type="ML"), dl_df.assign(type="DL")], ignore_index=True)
print("\n[BẢNG SO SÁNH TỔNG HỢP]")
print(all_df.sort_values(["acc", "type"], ascending=[False, True]))

# (e) Vẽ biểu đồ so sánh metrics
plt.figure(figsize=(10,5))
sns.barplot(data=all_df, x="model", y="acc", hue="type")
plt.title("Accuracy comparison (ML vs DL)")
plt.ylim(0,1); plt.show()

plt.figure(figsize=(10,6))
sns.barplot(data=all_df, x="model", y="mae", hue="type")
plt.title("MAE comparison"); plt.ylabel("MAE"); plt.xlabel("Model"); plt.show()

plt.figure(figsize=(10,6))
sns.barplot(data=all_df, x="model", y="mse", hue="type")
plt.title("MSE comparison"); plt.ylabel("MSE"); plt.xlabel("Model"); plt.show()
```

Hình 16



```
sns.barplot(data=all_df, x="model", y="mse", hue="type")
plt.title("MSE comparison"); plt.ylabel("MSE"); plt.xlabel("Model"); plt.show()

plt.figure(figsize=(10,6))
sns.barplot(data=all_df, x="model", y="rmse", hue="type")
plt.title("RMSE comparison"); plt.ylabel("RMSE"); plt.xlabel("Model"); plt.show()

# Learning curves
def plot_history(h, title):
    plt.figure(figsize=(6,4))
    plt.plot(h.history["accuracy"], label="train_acc")
    plt.plot(h.history["val_accuracy"], label="val_acc")
    plt.title(title); plt.xlabel("Epoch"); plt.ylabel("Accuracy"); plt.legend(); plt.show()
plot_history(hist_cnn, "CNN accuracy")
plot_history(hist_rnn, "RNN accuracy")
plot_history(hist_lstm, "LSTM accuracy")

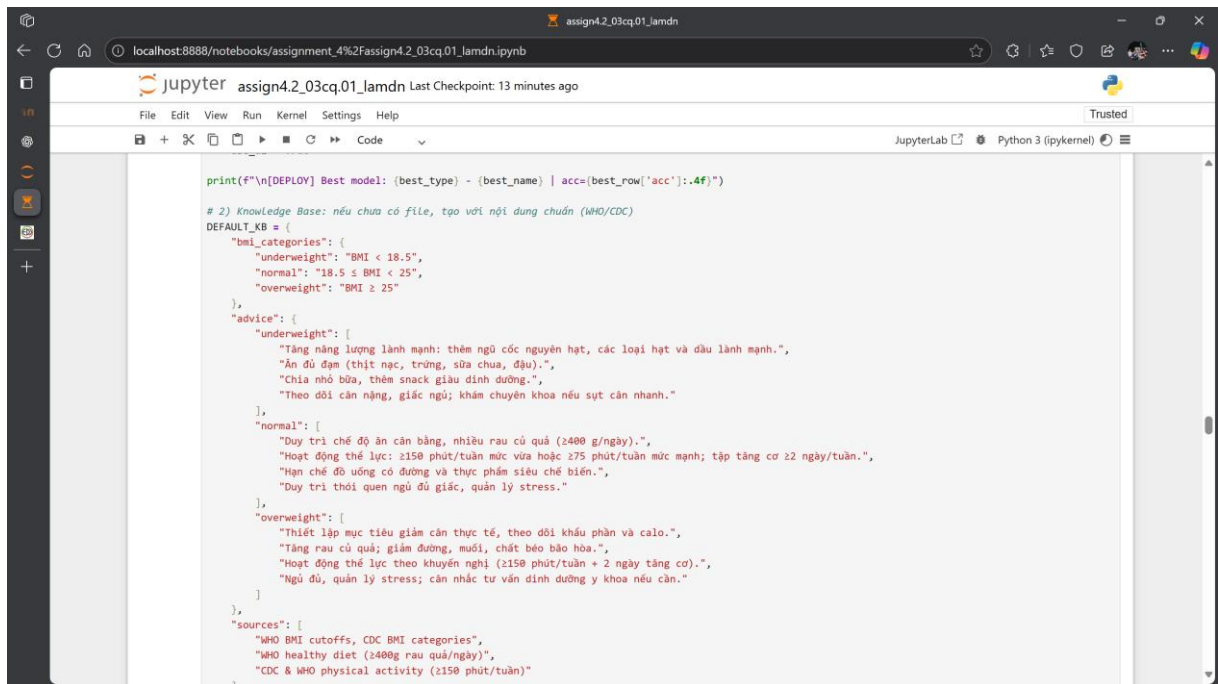
# -----
# (g) Deploy best model + Health Guide (KB JSON)
# -----

# 1) Chọn mô hình tốt nhất theo accuracy (ưu tiên DL, nếu bằng thì lấy DL)
best_row = all_df.sort_values(["acc", "type"], ascending=[False, True]).iloc[0]
best_name = best_row["model"]
best_type = best_row["type"]

if best_type == "ML":
    best_model = ml_pipelines[best_name]
    use_dl = False
else:
    m = {"CNN": cnn, "RNN": rnn, "LSTM": lstm}
    best_model = m[best_name]
    use_dl = True

print(f"Deploying best model: {best_model} (type: {best_type})")
```

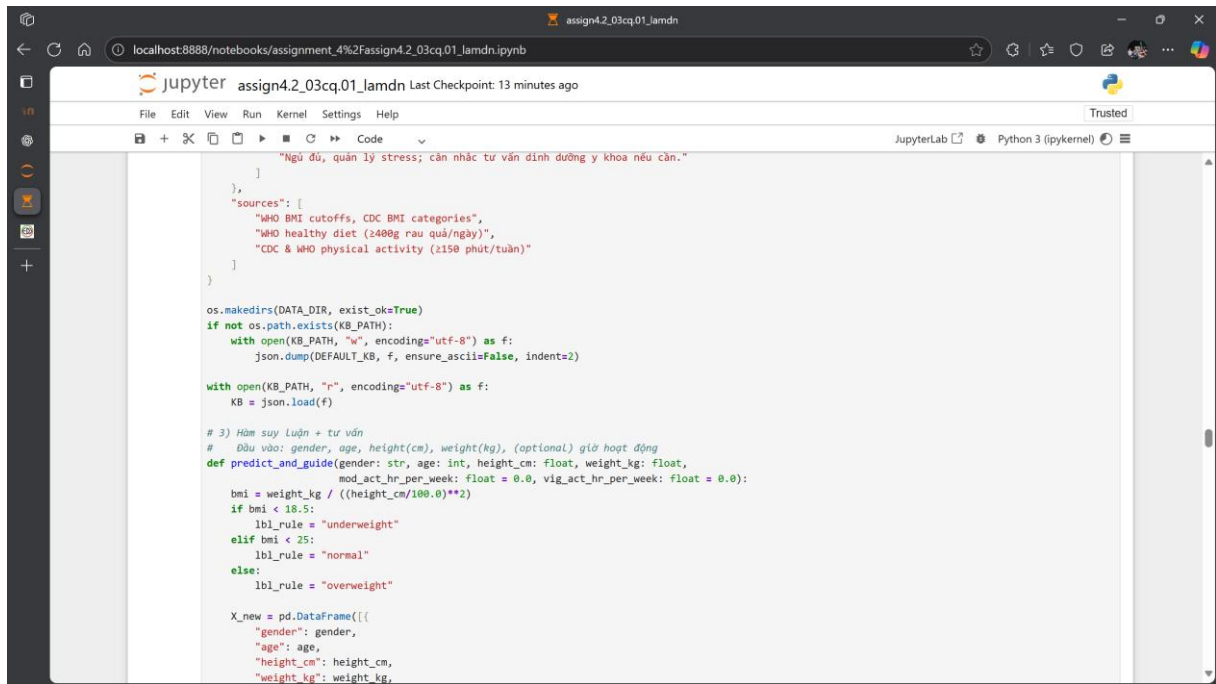
Hình 17



```
print(f"\n[DEPLOY] Best model: {best_type} - {best_name} | acc={best_row['acc']:.4f}")

# 2) Knowledge Base: nếu chưa có file, tạo với nội dung chuẩn (WHO/CDC)
DEFAULT_KB = {
    "bmi_categories": {
        "underweight": "BMI < 18.5",
        "normal": "18.5 ≤ BMI < 25",
        "overweight": "BMI ≥ 25"
    },
    "advice": {
        "underweight": [
            "Tăng năng lượng lành mạnh: thêm ngũ cốc nguyên hạt, các loại hạt và dầu lành mạnh.",
            "Ăn đủ đạm (thịt nạc, trứng, sữa chua, đậu).",
            "Chia nhỏ bữa, thêm snack giàu dinh dưỡng.",
            "Theo dõi cân nặng, giấc ngủ; khám chuyên khoa nếu sụt cân nhanh."
        ],
        "normal": [
            "Duy trì chế độ ăn cân bằng, nhiều rau củ quả (≥400 g/ngày).",
            "Hoạt động thể lực: ≥150 phút/tuần mức vừa hoặc 75 phút/tuần mức mạnh; tập tăng cơ 22 ngày/tuần.",
            "Hạn chế đồ uống có đường và thực phẩm siêu chế biến.",
            "Duy trì thói quen ngủ đủ giấc, quản lý stress."
        ],
        "overweight": [
            "Thiết lập mục tiêu giảm cân thực tế, theo dõi khẩu phần và calo.",
            "Tăng rau củ quả; giảm đường, muối, chất béo bão hòa.",
            "Hoạt động thể lực theo khuyến nghị (≥150 phút/tuần + 2 ngày tăng cơ).",
            "Ngủ đủ, quản lý stress; cân nhắc tư vấn dinh dưỡng y khoa nếu cần."
        ]
    },
    "sources": [
        "WHO BMI cutoffs, CDC BMI categories",
        "WHO healthy diet (≥400g rau quả/ngày)",
        "CDC & WHO physical activity (≥150 phút/tuần)"
    ]
}
```

Hình 18



```

    },
    "sources": [
        "WHO BMI cutoffs, CDC BMI categories",
        "WHO healthy diet (≥400g rau quả/ngày)",
        "CDC & WHO physical activity (≥150 phút/tuần)"
    ]
}

os.makedirs(DATA_DIR, exist_ok=True)
if not os.path.exists(KB_PATH):
    with open(KB_PATH, "w", encoding="utf-8") as f:
        json.dump(DEFAULT_KB, f, ensure_ascii=False, indent=2)

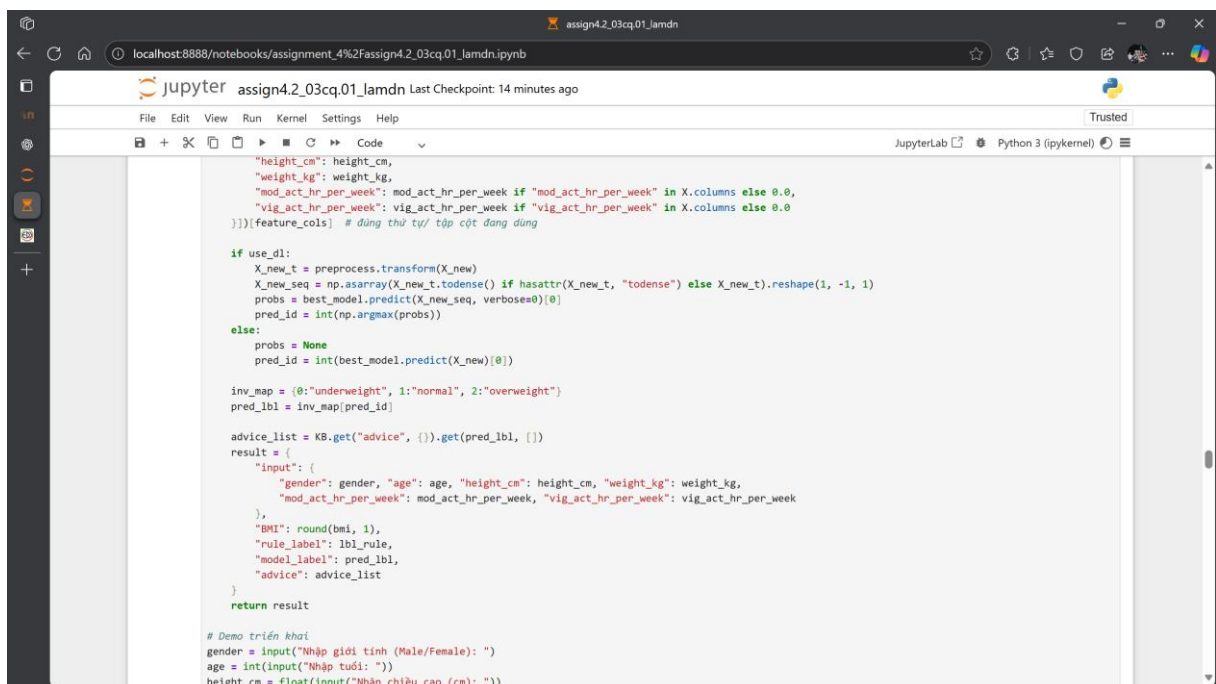
with open(KB_PATH, "r", encoding="utf-8") as f:
    KB = json.load(f)

# 3) Hàm suy luận + tư vấn
# Đầu vào: gender, age, height(cm), weight(kg), (optional) giờ hoạt động
def predict_and_guide(gender: str, age: int, height_cm: float, weight_kg: float,
                      mod_act_hr_per_week: float = 0.0, vig_act_hr_per_week: float = 0.0):
    bmi = weight_kg / ((height_cm/100.0)**2)
    if bmi < 18.5:
        lbl_rule = "underweight"
    elif bmi < 25:
        lbl_rule = "normal"
    else:
        lbl_rule = "overweight"

    X_new = pd.DataFrame([
        "gender": gender,
        "age": age,
        "height_cm": height_cm,
        "weight_kg": weight_kg,

```

Hình 19



```

    "height_cm": height_cm,
    "weight_kg": weight_kg,
    "mod_act_hr_per_week": mod_act_hr_per_week if "mod_act_hr_per_week" in X.columns else 0.0,
    "vig_act_hr_per_week": vig_act_hr_per_week if "vig_act_hr_per_week" in X.columns else 0.0
  ]][feature_cols] # dùng thứ tự/ tập cột đang dùng

if use_d1:
    X_new_t = preprocess.transform(X_new)
    X_new_seq = np.asarray(X_new_t.todense() if hasattr(X_new_t, "todense") else X_new_t.reshape(1, -1, 1))
    probs = best_model.predict(X_new_seq, verbose=0)[0]
    pred_id = int(np.argmax(probs))
else:
    probs = None
    pred_id = int(best_model.predict(X_new)[0])

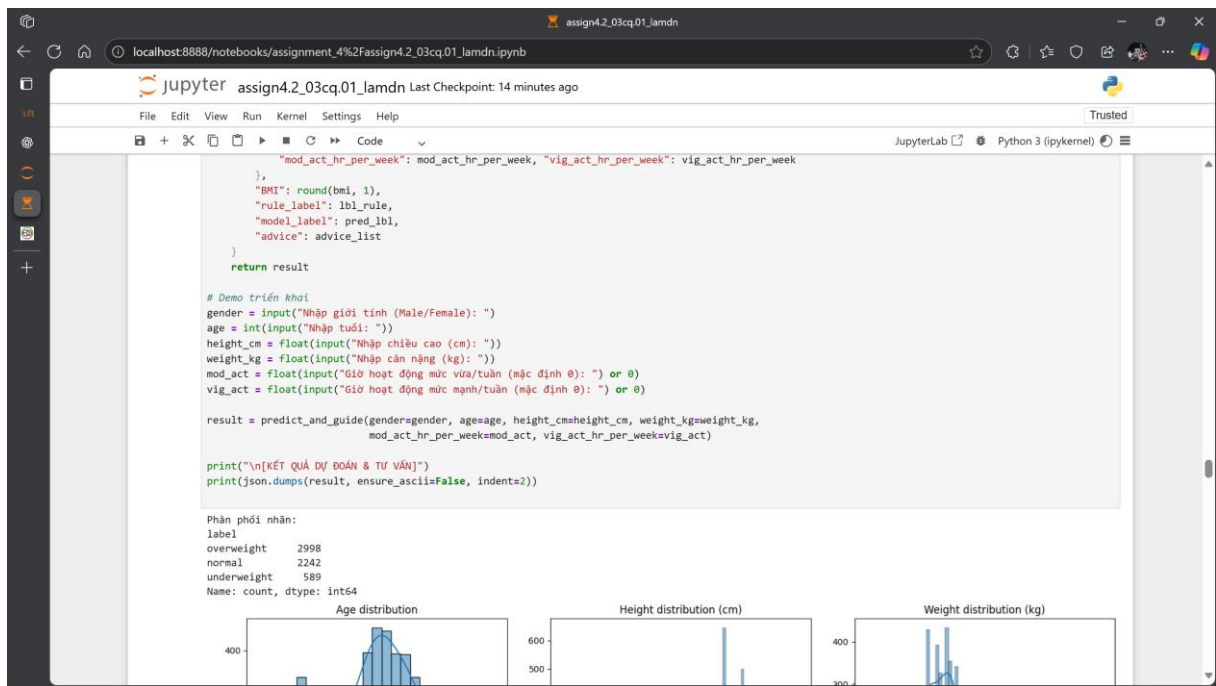
inv_map = {0:"underweight", 1:"normal", 2:"overweight"}
pred_lbl = inv_map[pred_id]

advice_list = KB.get("advice", {}).get(pred_lbl, [])
result = {
    "input": {
        "gender": gender, "age": age, "height_cm": height_cm, "weight_kg": weight_kg,
        "mod_act_hr_per_week": mod_act_hr_per_week, "vig_act_hr_per_week": vig_act_hr_per_week
    },
    "BMI": round(bmi, 1),
    "rule_label": lbl_rule,
    "model_label": pred_lbl,
    "advice": advice_list
}
return result

# Demo triển khai
gender = input("Nhập giới tính (Male/Female): ")
age = int(input("Nhập tuổi: "))
height_cm = float(input("Nhập chiều cao (cm): "))

```

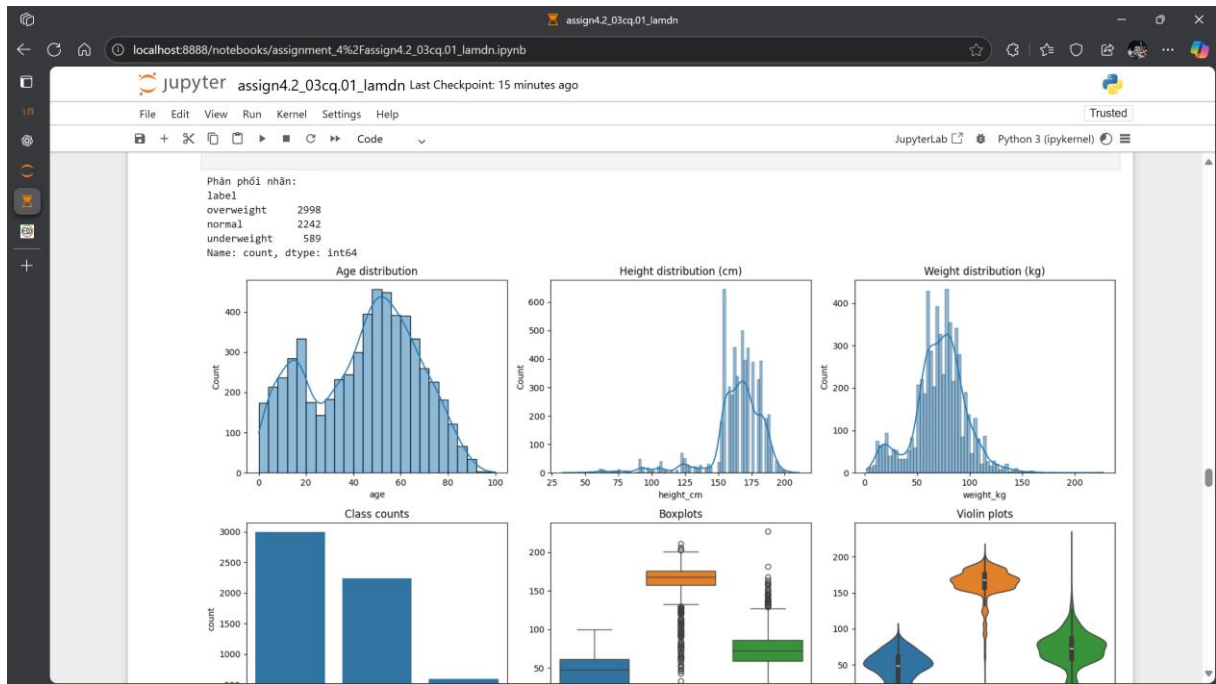
Hình 20



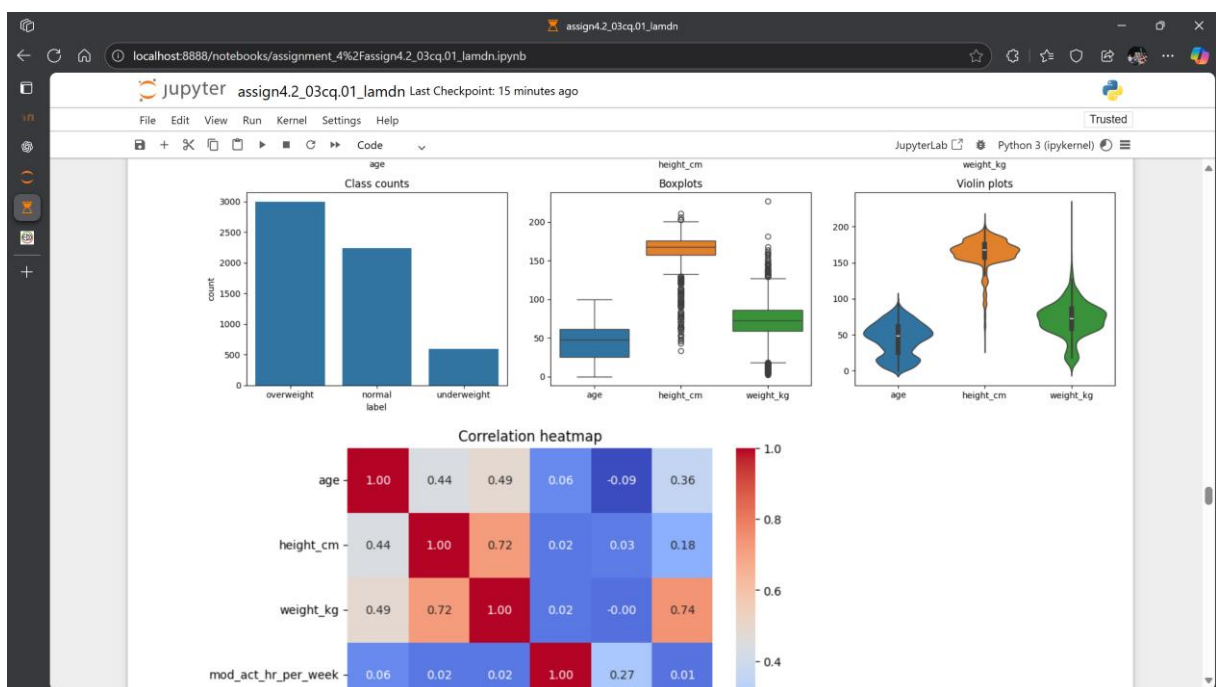
Hình 21



## ⇒ Kết quả

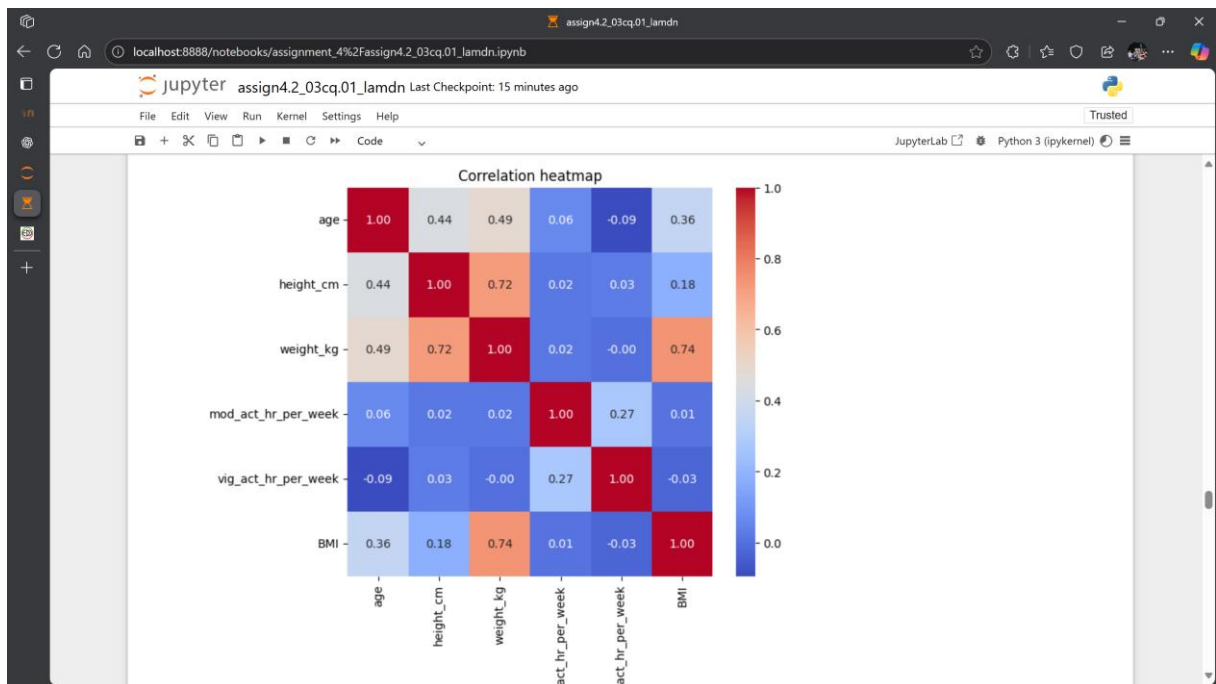


Hình 22

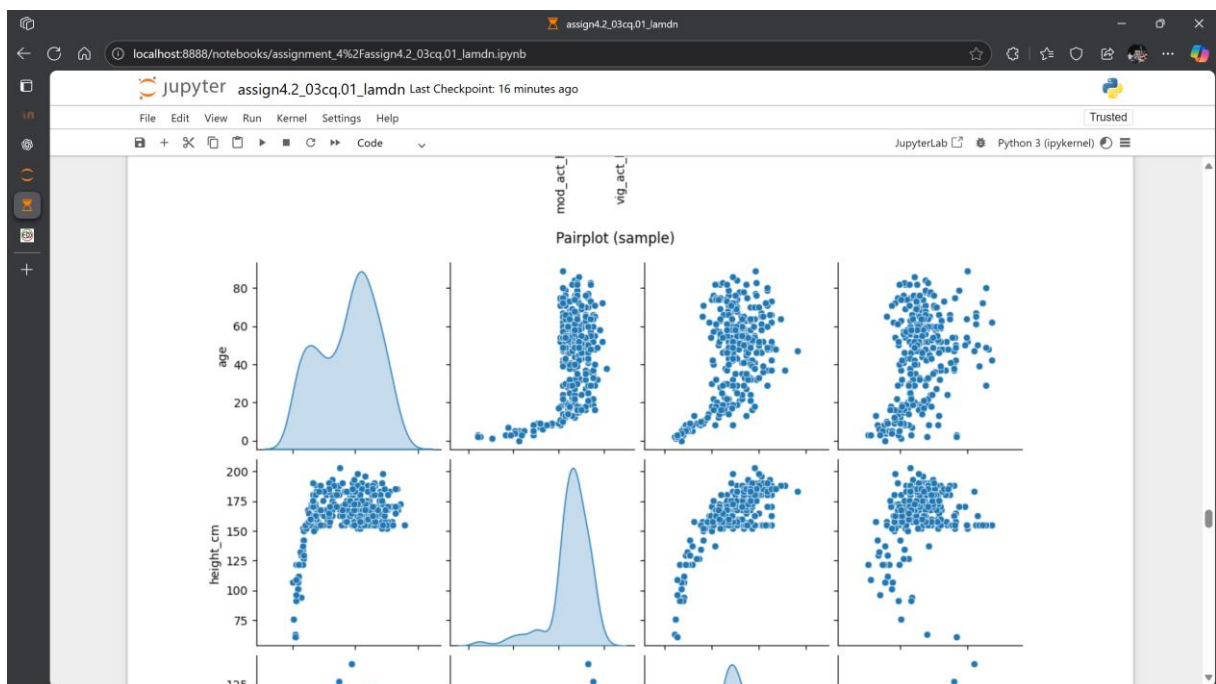


Hình 23





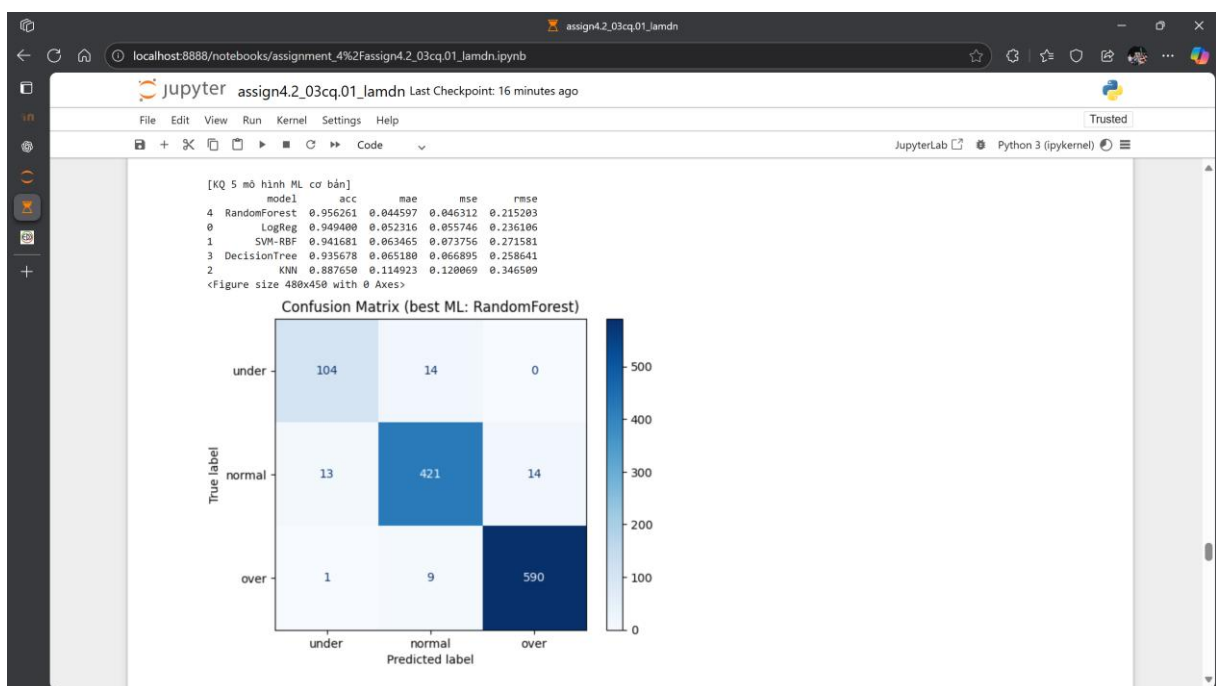
Hình 24



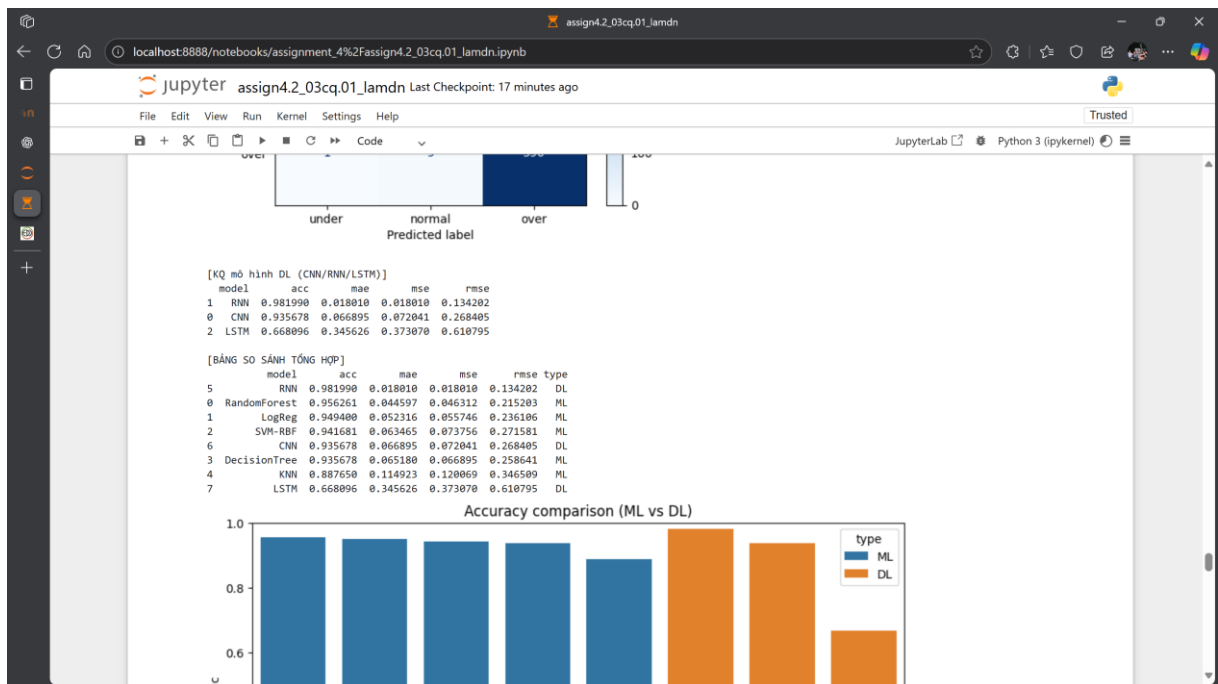
Hình 25



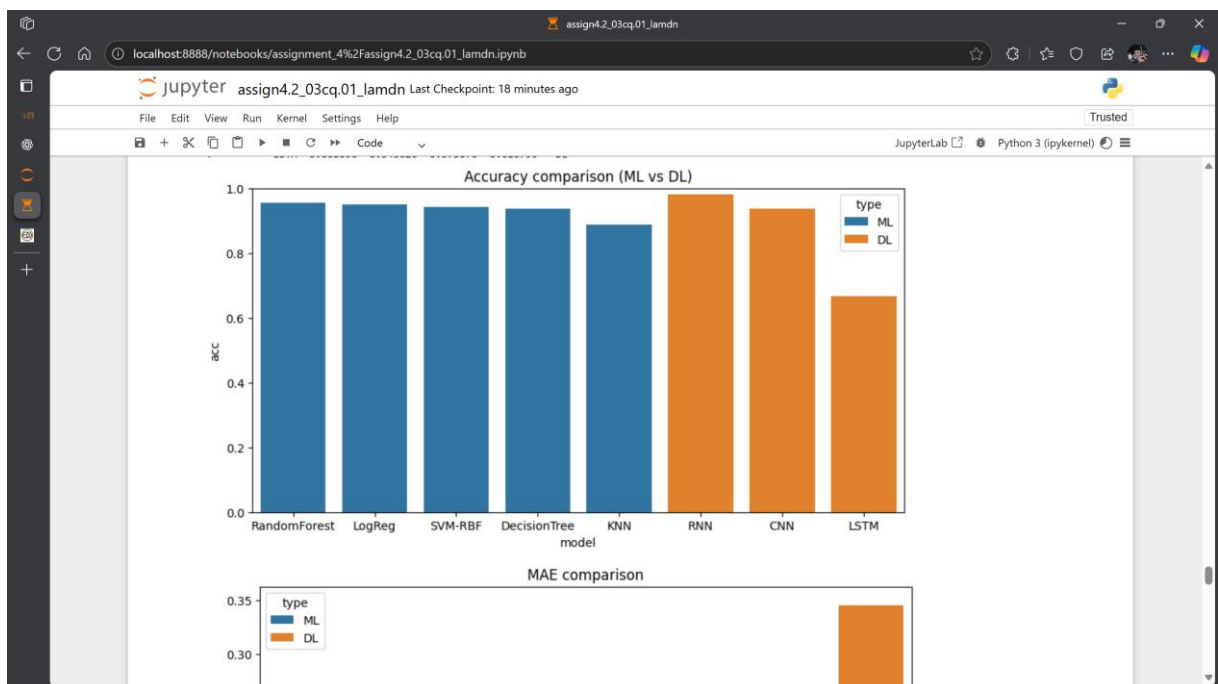
Hình 26



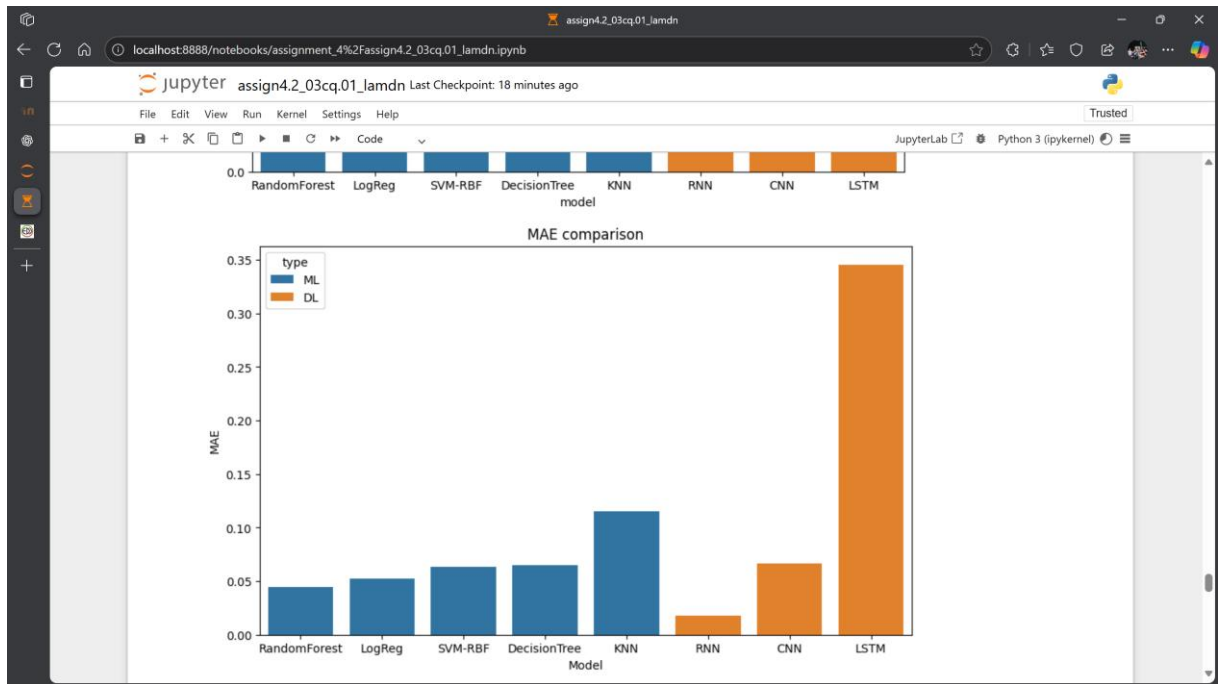
Hình 27



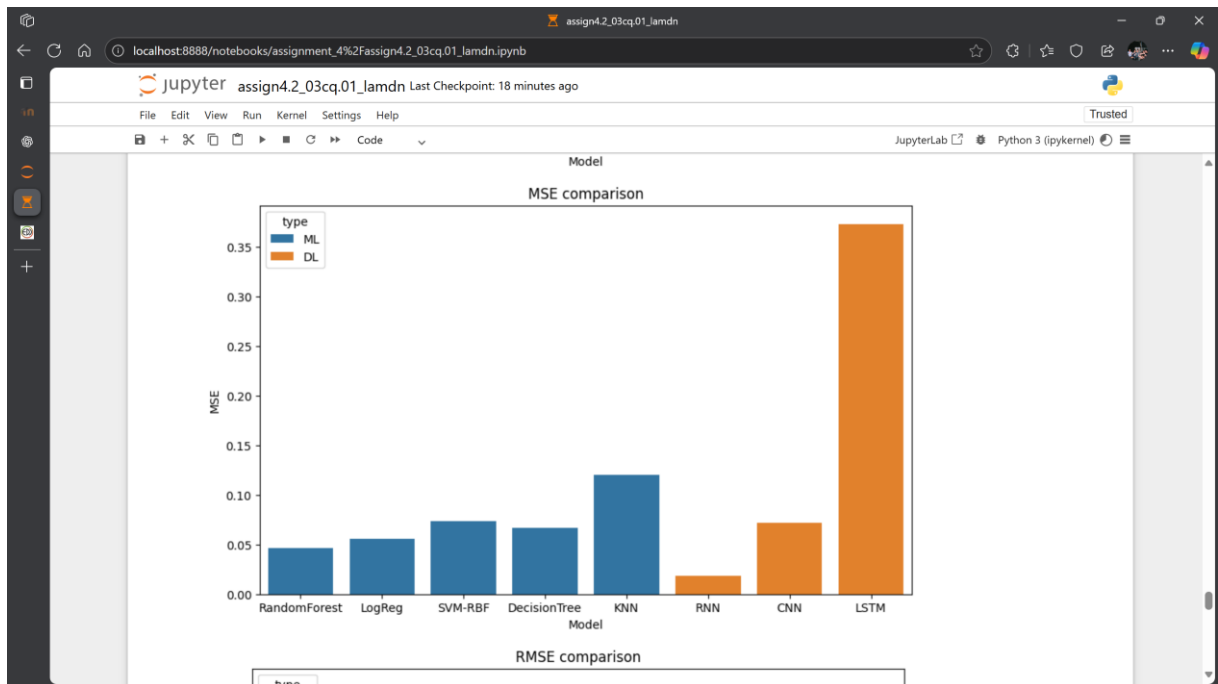
Hình 28



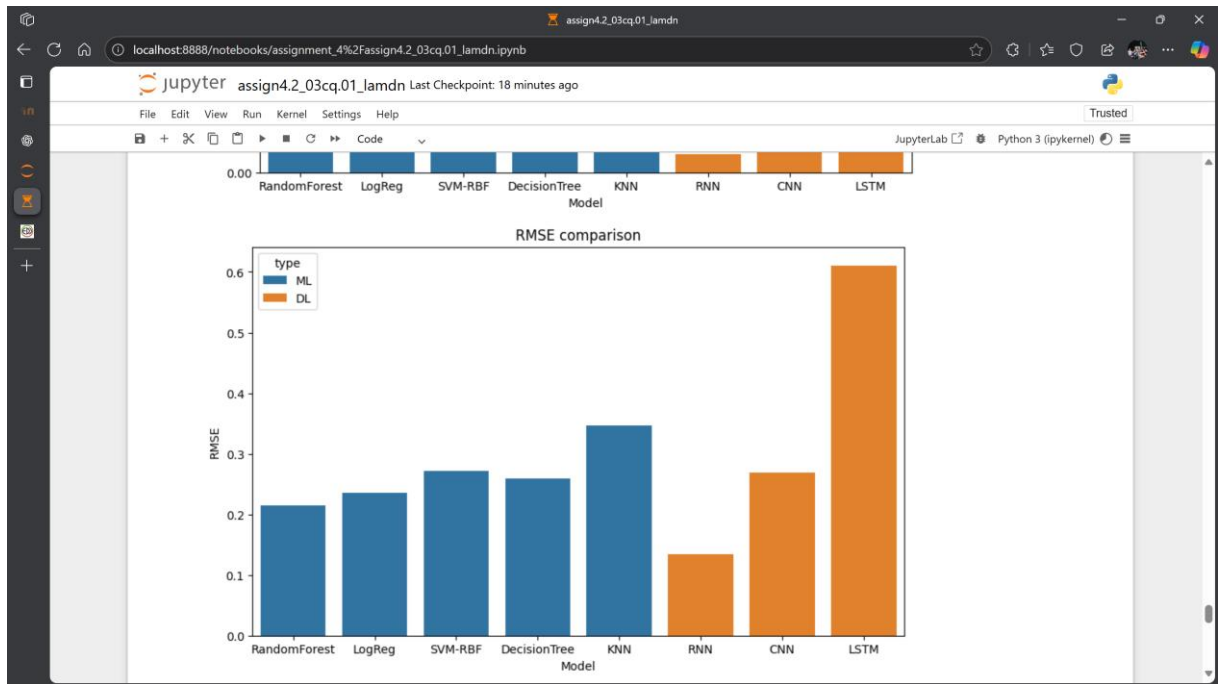
Hình 29



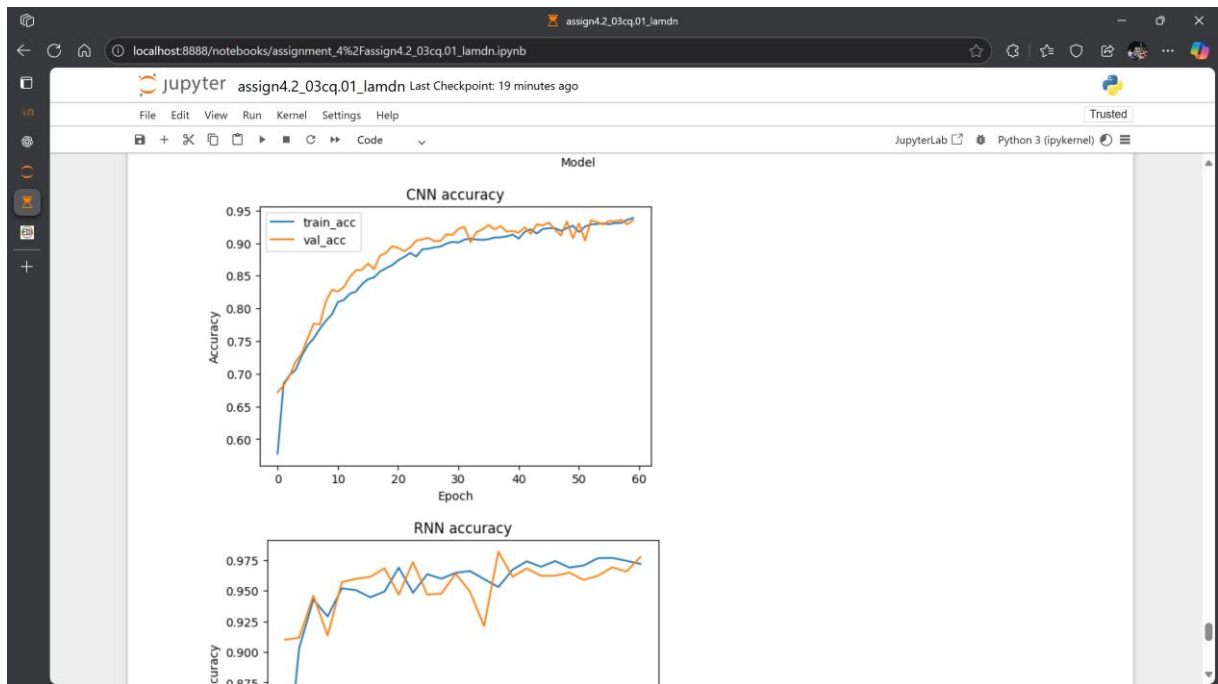
Hình 30



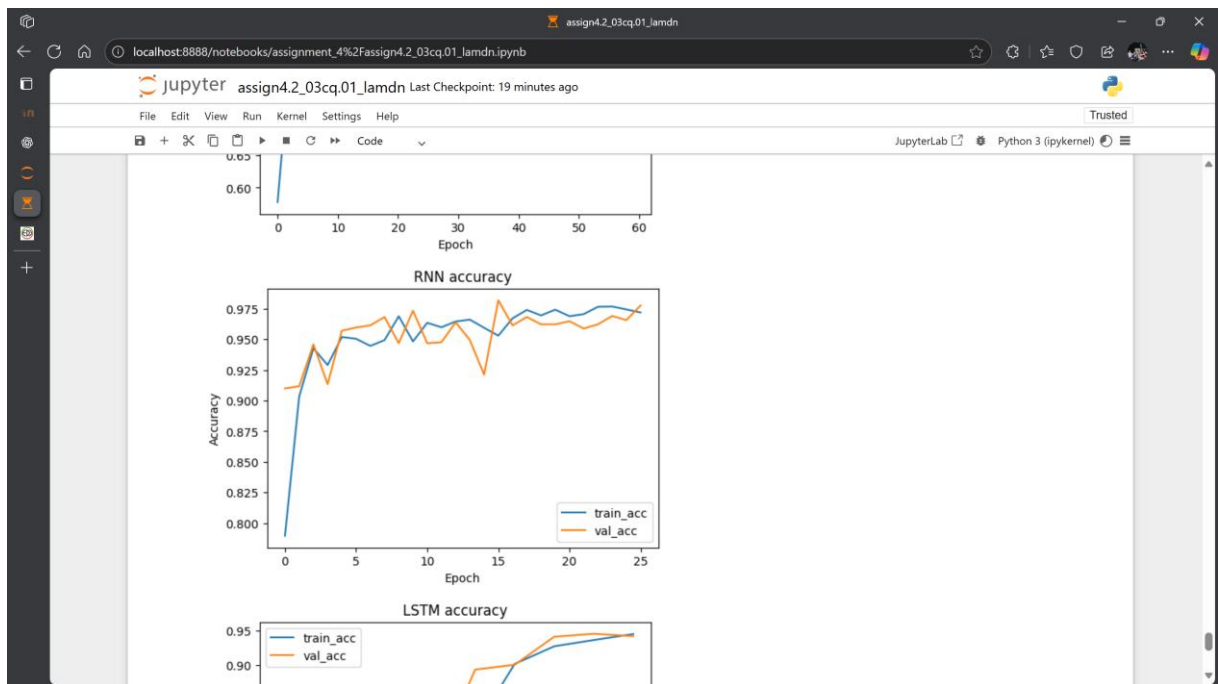
Hình 31



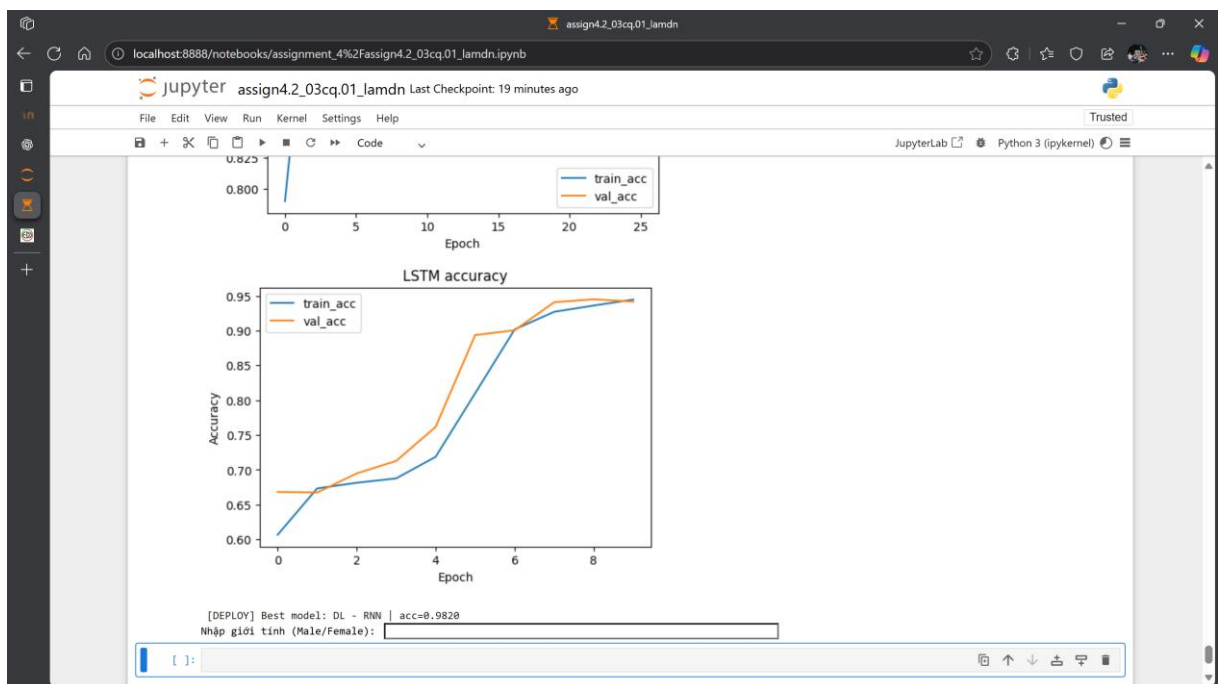
Hình 32



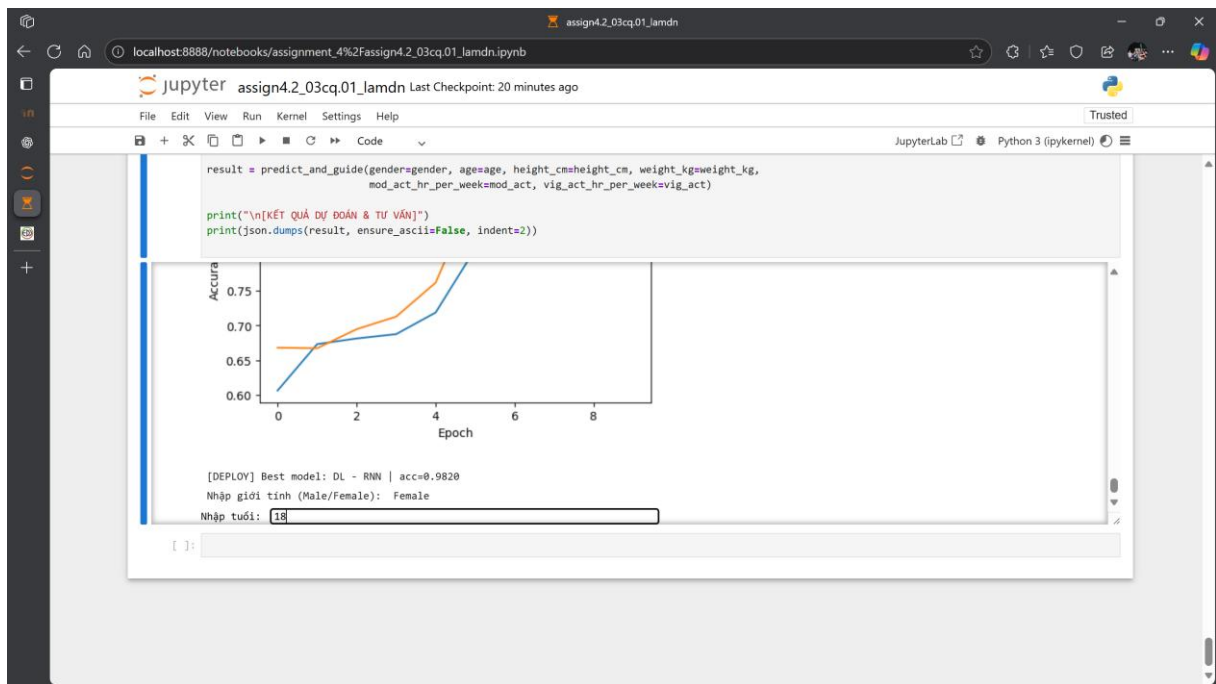
Hình 33



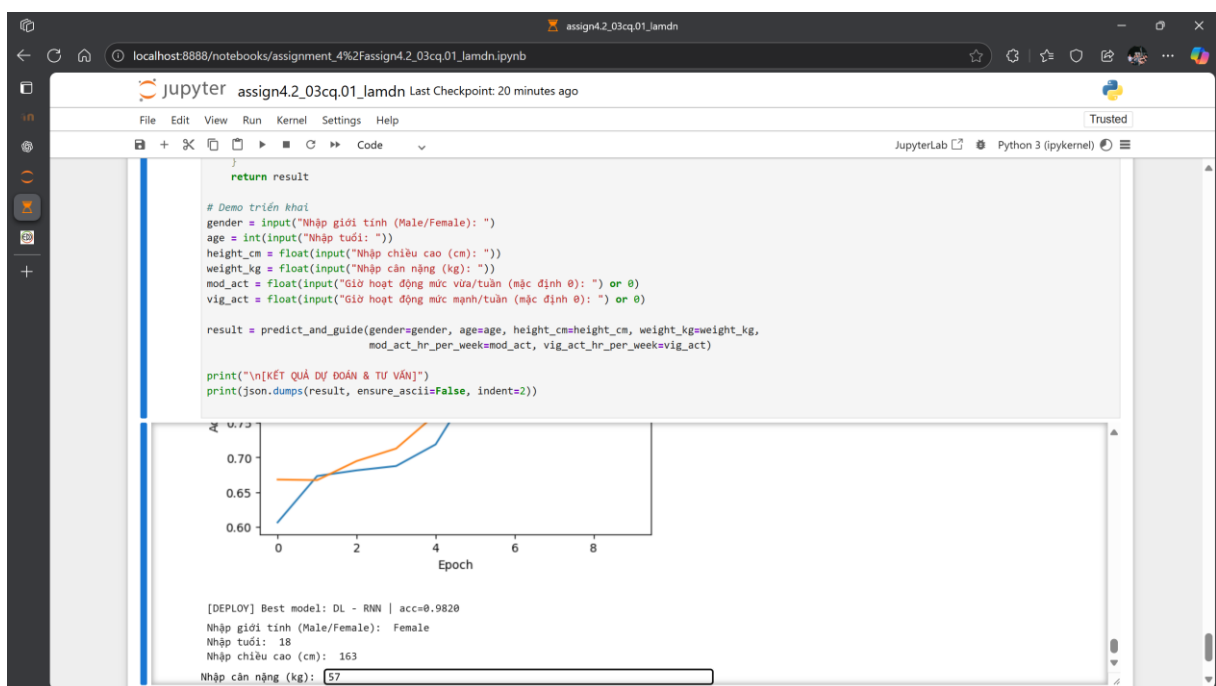
Hình 34



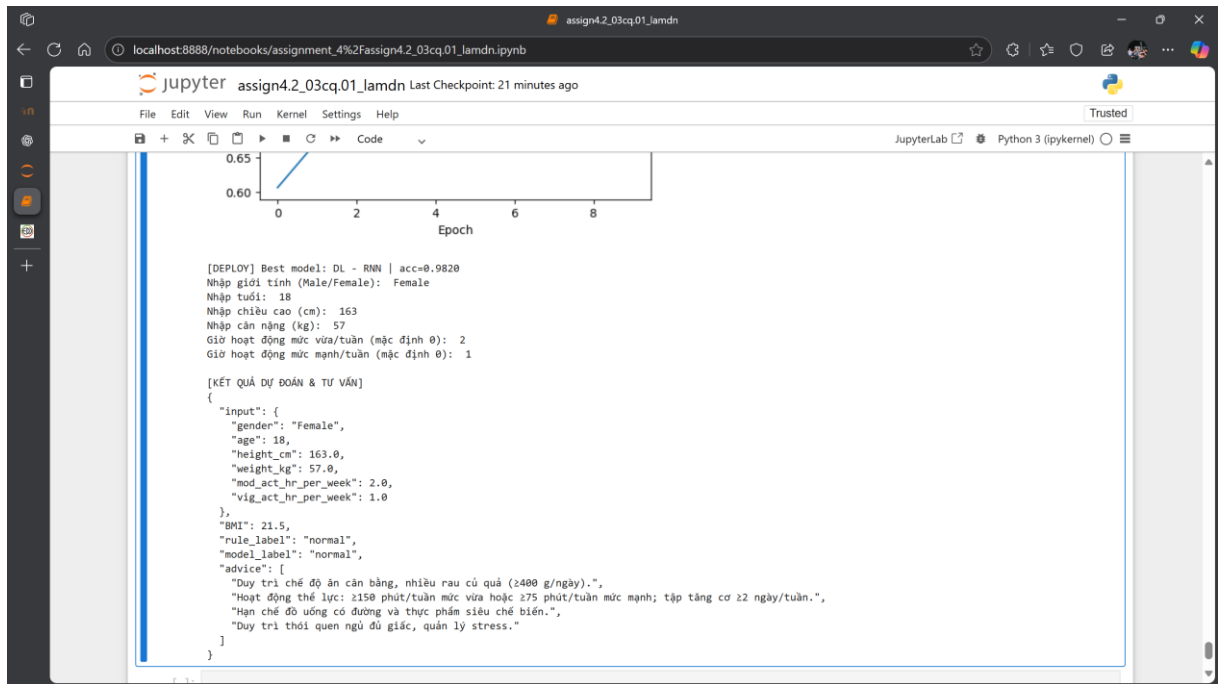
Hình 35



Hình 36



Hình 37



Hình 38